

optical-metrology-fitter

코드 전문과 물리 해설

전달행렬법 기반 박막 광학 계측 —
순방향 모델, 역문제, 그리고 불확도 예산

규약 $e^{-i\omega t}$, $\tilde{n} = n + ik$ ($k \geq 0$), 순방향파 $e^{+ik_z z}$

핵심 결과 알고리즘 잡음 바닥 ≈ 4 pm vs 현실적 정확도 ≈ 1.3 nm (격차 $\approx 300\times$)

검증 34/34 통과, Byrnes 레퍼런스 구현체와 10^{-14} 수준 일치

이 문서는 저장소의 모든 소스 코드를, 각 코드 덩어리 바로 옆(또는 아래)에 대응하는 물리적 근거를 붙여 재구성한 것입니다. Part I은 코드와 독립적인 물리 유도, Part II는 주석 달린 소스, Part III는 축약 없는 전체 원본입니다.

2026년 8월

Contents

1 읽기 전에 — 이 문서의 구조	3
Part I — 물리	3
2 Fresnel 계수 — 사실은 임피던스 정합	3
2.1 왜 경계조건에서 출발하는가	3
2.2 s편광 (TE): $\mathbf{E} \parallel \hat{y}$	3
2.3 p편광 (TM): $\mathbf{H} \parallel \hat{y}$	4
2.4 s와 p의 통합 — 광학 어드미턴스	4
2.5 Brewster 각 — 쌍극자 복사 패턴	4
3 위상 규약과 분지 선택	4
3.1 세 선택은 독립이 아니다	4
3.2 스넬 법칙을 각도로 쓰지 않은 이유	5
4 다중 반사 — Airy 급수와 전달행렬은 같은 것	5
4.1 경로 합	5
4.2 Stokes 관계 — 시간 역전 대칭의 산물	5
4.3 전달행렬은 이 급수를 미리 합해둔 것	5
4.4 반파장층은 왜 투명한가	6
5 투과율 — p편광에서 켈레가 필요한 이유	6
5.1 반무한 흡수 기판에서 T 의 의미	6
6 물질 — 분산은 어디서 오는가	6
6.1 SiO_2 : Sellmeier는 감쇠 없는 Lorentz 진동자	6
6.2 Si: 왜 Sellmeier가 안 되는가	7
7 역문제 — 왜 어려운가	7
7.1 근본 축퇴	7
7.2 $n \cdot d$ 축퇴 — 반사계의 원죄	7
8 정밀도의 한계 — Fisher 정보량과 Cramér-Rao 하한	8
8.1 유도	8
8.2 물리적 읽기	8
8.3 가장 중요한 비대칭	8
8.4 계통오차의 물리	8
8.5 최종 예산	9
Part II — 주석 달린 소스	10
9 omf/core.py — 순방향 모델	10
9.1 규약 선언과 분지 상수	10
9.2 분지 선택 — 이 코드의 유일한 물리적 강제	10
9.3 면내 파수 보존 = 스넬 법칙	10
9.4 Fresnel 계수	11
9.5 전달행렬 조립 — 함수의 골격	11

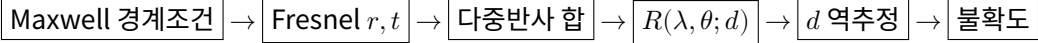
9.6	위상과 계면 항 — 벡터화가 일어나는 지점	12
9.7	층 행렬과 누적 곱	12
9.8	반사율과 투과율 — p편광의 켈레	13
10	omf/materials.py — 굴절률 공급	13
10.1	Sellmeier — 감쇠 없는 Lorentz 진동자	13
10.2	비분산 매질과 상수 공장	13
10.3	Si — 표 보간과 로그 스케일	14
10.4	두 번째 Si 데이터셋 — 계통오차를 재기 위한 장치	15
11	omf/fitter.py — 역문제	15
11.1	모듈 서두 — 왜 단순 최소제곱이 실패하는가	15
11.2	FitResult — 불확도를 결과에 동봉	16
11.3	FilmModel 생성자 — 무엇을 자유 파라미터로 둘 것인가	16
11.4	순방향 래퍼	17
11.5	조밀 격자 사전탐색 — 전역성 확보	17
11.6	정밀화와 공분산	18
12	omf/uncertainty.py — 판정	18
12.1	모듈 서두 — 왜 CRLB인가	18
12.2	수치 야코비안과 Fisher 정보량	19
12.3	몬테카를로 — 그리고 inverse crime의 명시	20
13	실험 스크립트가 조작하는 손잡이	20
13.1	inverse crime을 깨는 코드	20
Part III	— 전체 원본 소스	20
14	축약 없는 소스 코드	20
14.1	omf/core.py — TMM 엔진 (260줄)	21
14.2	omf/materials.py — 분산 모델 (191줄)	23
14.3	omf/fitter.py — 역문제 솔버 (222줄)	26
14.4	omf/uncertainty.py — 불확도 분석 (105줄)	28
14.5	omf/_init_.py — 패키지 인터페이스 (22줄)	29
14.6	tests/test_core.py — 검증 스위트 (34 항목) (416줄)	30
14.7	scripts/exp01_forward_model.py — 실험 1 — 순방향 모델 (181줄)	34
14.8	scripts/exp02_inverse_ambiguity.py — 실험 2 — 역문제와 축퇴 (223줄)	36
14.9	scripts/exp03_precision_crlb.py — 실험 3 — CRLB 대 몬테카를로 (241줄)	39
14.10	scripts/exp04_systematics.py — 실험 4 — 계통오차 예산 (312줄)	42
14.11	scripts/_style.py — 그림 스타일 (59줄)	45
14.12	run_all.py — 전체 실행기 (52줄)	46
15	맺음말 — 이 문서에서 반복되는 하나의 주제	46

1 읽기 전에 — 이 문서의 구조

이 프로젝트가 답하려는 질문은 하나입니다.

반사율 R 을 측정했을 때, 박막 두께 d 를 얼마나 믿을 수 있는가?

인과 사슬은 다음과 같고, 문서 전체가 이 순서를 따릅니다.



핵심 명제를 미리 못 박아 둡니다. **박막 두께는 직접 관측량이 아닙니다.** 관측량은 왕복 위상 2δ 하나뿐이고 두께는 거기서 추론됩니다. 이 프로젝트의 모든 어려움 — 프린지 차수 측퇴, $n \cdot d$ 측퇴, 300배 격차 — 이 전부 이 한 문장에서 파생됩니다.

문서 구성

- **Part I (§ 2–§ 7)** — 코드와 무관하게 성립하는 물리. Maxwell 경계조건에서 Cramér–Rao 하한까지.
- **Part II (§ 8–§ 12)** — 소스 코드 전문. 코드 덩어리마다 바로 아래에 파란 세로선으로 표시된 물리 해설을 붙였습니다. 표시된 줄 번호는 각 모듈 원본 파일의 실제 줄 번호가 아니라 덩어리 내부의 상대 번호입니다.
- **Part III (§ 13)** — 축약 없는 전체 원본 소스. Part II에서 지면상 접었던 docstring까지 전부 포함합니다.

Part II의 표기 약속. 코드 덩어리는 회색 배경에 파란 줄번호로, 물리 해설은 파란 세로선 상자로 구분됩니다. Part II의 코드 덩어리에서는 여러 줄 docstring을 첫 줄로 접었습니다 — 그 내용이 곧 옆의 해설이기 때문입니다. 접지 않은 완전한 원본은 Part III에 있습니다.

Part I — 물리

2 Fresnel 계수 — 사실은 임피던스 정합

2.1 왜 경계조건에서 출발하는가

자유전하·자유전류가 없는 계면에 Maxwell 방정식의 적분형을 적용하면 단 두 개의 조건이 남습니다.

$$\hat{n} \times (\mathbf{E}_2 - \mathbf{E}_1) = 0, \quad \hat{n} \times (\mathbf{H}_2 - \mathbf{H}_1) = 0 \quad (1)$$

즉 **접선 성분** E, H 의 연속입니다. 이것이 전부이고, 나머지는 순수한 대수입니다. 여기서 유의할 점은 **D**와 **B**의 법선 성분 연속은 위 두 조건의 종속 결과라는 것입니다 — 독립 조건이 아니므로 따로 쓸 필요가 없습니다.

2.2 s편광 (TE): $\mathbf{E} \parallel \hat{y}$

E_y 연속으로부터

$$1 + r = t \quad (2)$$

$\mathbf{H} = (\mathbf{k} \times \mathbf{E})/(\omega\mu_0)$ 이므로 $H_x \propto k_z E_y$ 이고, H_x 연속으로부터

$$k_{z1}(1 - r) = k_{z2}t \quad (3)$$

두 식을 연립하면

$$r_s = \frac{k_{z1} - k_{z2}}{k_{z1} + k_{z2}}, \quad t_s = \frac{2k_{z1}}{k_{z1} + k_{z2}} \quad (4)$$

직관. 이 식은 전송선 임피던스 정합 공식 $\Gamma = (Z_2 - Z_1)/(Z_2 + Z_1)$ 과 글자 그대로 같은 형태입니다. k_z 가 광학 어드미턴스 역할을 합니다. 임피던스가 맞으면 반사가 없다는 전기공학의 직관이 그대로 광학에 이식됩니다.

2.3 p편광 (TM): $\mathbf{H} \parallel \hat{y}$

H_y 연속으로부터 $1 + r = t_H$. 그리고 $\nabla \times \mathbf{H} = -i\omega\epsilon\mathbf{E}$ 에서 $E_x \propto (k_z/\epsilon)H_y$ 이므로

$$\frac{k_{z1}}{\epsilon_1}(1 - r) = \frac{k_{z2}}{\epsilon_2}t_H \quad (5)$$

따라서

$$r_p = \frac{\epsilon_2 k_{z1} - \epsilon_1 k_{z2}}{\epsilon_2 k_{z1} + \epsilon_1 k_{z2}} = \frac{\tilde{n}_2^2 k_{z1} - \tilde{n}_1^2 k_{z2}}{\tilde{n}_2^2 k_{z1} + \tilde{n}_1^2 k_{z2}} \quad (6)$$

코드의 $t_p = 2\tilde{n}_1\tilde{n}_2 k_{z1}/(\tilde{n}_2^2 k_{z1} + \tilde{n}_1^2 k_{z2})$ 는 위 결과를 H 진폭이 아니라 E **진폭**으로 환산한 것입니다 ($|E| = |H|/\tilde{n}$).

2.4 s와 p의 통합 — 광학 어드미턴스

$$r = \frac{Y_1 - Y_2}{Y_1 + Y_2}, \quad Y_s \propto k_z, \quad Y_p \propto \frac{\tilde{n}^2}{k_z} \quad (7)$$

s와 p는 다른 물리가 아니라 **다른 어드미턴스**입니다. 박막 설계 문헌 전체가 이 형식으로 쓰여 있습니다.

함정 — r_p 의 부호는 규약입니다. Y_p 정의로 계산하면 이 프로젝트의 r_p 와 부호가 반대로 나옵니다. 이는 반사 후 $\mathbf{E}_p$ 의 양의 방향을 어떻게 잡느냐의 순수한 규약 문제입니다. $|r_p|^2$ 는 규약과 무관하므로 R_p 는 안전하지만, 서로 다른 출처의 r_p 정의를 한 계산에 섞으면 반드시 사고가 납니다. 이 저장소는 Byrnes 규약(θ_B 아래에서 $r_p > 0$)을 일관되게 씁니다.

2.5 Brewster 각 — 쌍극자 복사 패턴

$\theta_1 + \theta_2 = 90^\circ$ 일 때 $r_p = 0$ 이고, 이때 $\tan \theta_B = n_2/n_1$ 입니다. 물리적 이유는 다음과 같습니다.

매질 2에서 유도된 쌍극자는 투과파의 $\mathbf{E}_p$ 방향으로 진동합니다. p편광에서 이 방향은 반사파가 나가야 할 방향과 정확히 평행해집니다. 그런데 쌍극자 복사는 $\propto \sin^2 \Theta$ 이므로 **자기 축 방향으로의 복사는 없습니다**. 따라서 반사파가 아예 생성되지 않습니다.

흡수가 있으면 왜 0이 안 되는가. $k \neq 0$ 이면 쌍극자 응답에 위상 지연이 생겨 상쇄가 불완전해집니다. 실험 1에서 405 nm의 Si는 $R_{p,\min} = 5.2 \times 10^{-4}$ 인 반면, 무손실 BK7은 4×10^{-8} 입니다. **이 잔류량 자체가 k 의 직접 측정량**이며, 엘립소메트리의 물리적 근거입니다.

3 위상 규약과 분지 선택

3.1 세 선택은 독립이 아니다

선택	이 프로젝트
시간 의존성	$e^{-i\omega t}$
복소 굴절률	$\tilde{n} = n + ik, k \geq 0$
순방향파	$e^{+ik_z z}$

$+z$ 진행파의 진폭을 전개하면

$$e^{ik_z z} = \underbrace{e^{i\operatorname{Re}(k_z)z}}_{\text{진동}} \cdot \underbrace{e^{-\operatorname{Im}(k_z)z}}_{\text{감쇠}} \quad (8)$$

수동 매질에서 감쇠하려면 $\operatorname{Im}(k_z) \geq 0$ 이어야 합니다. **이것이 코드가 필요로 하는 유일한 분지 규칙입니다.**

반대 분지를 고르면 (i) 거리에 따라 증폭되는 비물리적 파가 되고, (ii) 두꺼운 흡수층에서 $e^{ik_z d}$ 가 오버플로를 일으켜 계산이 터집니다.

규약 혼용 경고. 엘립소메트리 문헌의 상당수는 $e^{+i\omega t}$, $\tilde{n} = n - ik$ 를 씁니다. 두 규약 모두 옳지만 섞으면 반드시 틀립니다. 그래서 이 저장소는 core.py 최상단에 규약을 명시하고 이후 어디서도 다시 건드리지 않습니다.

3.2 스넬 법칙을 각도로 쓰지 않은 이유

면내 병진 대칭성으로부터 면내 파수가 보존됩니다.

$$k_x = n_0 k_0 \sin \theta_0 = \text{const}, \quad k_{z,j} = \sqrt{(\tilde{n}_j k_0)^2 - k_x^2} \quad (9)$$

이것이 **스넬 법칙 그 자체**입니다. $\theta_j = \arcsin(\cdot)$ 로 쓰는 대신 k_z 로 쓰면 다음이 자동으로 처리됩니다.

- 전반사 $\rightarrow k_z$ 가 순허수
- 흡수 매질 $\rightarrow k_z$ 가 복소수
- 소멸파(frustrated TIR) $\rightarrow$ 위와 동일

각도로 쓰면 복소 $\arcsin$ 이 어느 리만 시트에 있는지를 매번 추적해야 합니다. **특수 케이스가 하나도 없는 코드**가 나온 이유가 이것입니다.

4 다중 반사 — Airy 급수와 전달행렬은 같은 것

4.1 경로 합

박막 내부에서 빛이 무한히 왕복하며 매 왕복마다 위상 2δ 를 얻습니다.

$$\delta = k_{z1} d = \frac{2\pi}{\lambda} \tilde{n}_1 d \cos \theta_1 \quad (10)$$

전체 반사 진폭은 모든 경로의 코히런트 합입니다.

$$r = r_{01} + t_{01} r_{12} t_{10} e^{2i\delta} + t_{01} r_{12} r_{10} r_{12} t_{10} e^{4i\delta} + \dots \quad (11)$$

등비급수를 합하면

$$r = r_{01} + \frac{t_{01} t_{10} r_{12} e^{2i\delta}}{1 - r_{10} r_{12} e^{2i\delta}} \quad (12)$$

4.2 Stokes 관계 — 시간 역전 대칭의 산물

무손실 계면에서 시간 역전 불변성으로부터

$$t_{01} t_{10} = 1 - r_{01}^2, \quad r_{10} = -r_{01} \quad (13)$$

이를 대입하면

$$r = \frac{r_{01} + r_{12} e^{2i\delta}}{1 + r_{01} r_{12} e^{2i\delta}} \quad (\text{Airy 공식}) \quad (14)$$

4.3 전달행렬은 이 급수를 미리 합해둔 것

$$M = D_{01} \prod_{j=1}^{N-2} P_j D_{j,j+1}, \quad D_{ij} = \frac{1}{t_{ij}} \begin{pmatrix} 1 & r_{ij} \\ r_{ij} & 1 \end{pmatrix}, \quad P_j = \begin{pmatrix} e^{-i\delta_j} & 0 \\ 0 & e^{+i\delta_j} \end{pmatrix} \quad (15)$$

$$r = \frac{M_{21}}{M_{11}}, \quad t = \frac{1}{M_{11}} \quad (16)$$

$N = 3$ 에 대해 직접 곱하면

$$M_{11} = \frac{e^{-i\delta} + r_{01} r_{12} e^{i\delta}}{t_{01} t_{12}}, \quad M_{21} = \frac{r_{01} e^{-i\delta} + r_{12} e^{i\delta}}{t_{01} t_{12}} \quad (17)$$

이므로 $r = M_{21}/M_{11}$ 이 정확히 Airy 공식으로 환원됩니다 (검증 B5, 오차 2×10^{-16}).

행렬 형식의 이점. 층이 N 개여도 급수를 다시 유도할 필요 없이 행렬 곱만 늘리면 됩니다. 그리고 D (계면, 진폭 섞기)와 P (전파, 위상 축적)로 물리적 역할이 깔끔히 분리됩니다. 이 분리가 코드 구조에 그대로 반영되어 있습니다.

4.4 반파장층은 왜 투명한가

$\delta = \pi$ 이면 $e^{2i\delta} = 1$ 이므로

$$r = \frac{r_{01} + r_{12}}{1 + r_{01}r_{12}} = r_{02} \quad (18)$$

막이 없는 것과 완전히 동일합니다. 왕복 위상이 정확히 2π 라 모든 경로가 동상으로 재결합해 계면 하나짜리 결과로 붕괴합니다. 검증 B6에서 $d_{1/2} = 226.917 \text{ nm}$ 에 대해 오차 1×10^{-16} 로 확인했습니다.

5 투과율 — p편광에서 켈레가 필요한 이유

T 는 $|t|^2$ 가 아니라 z 방향 시간평균 Poynting 플럭스의 비입니다.

$$S_z = \frac{1}{2} \text{Re}(\mathbf{E} \times \mathbf{H}^*)_z \quad (19)$$

s편광: $H_x \propto k_z E_y$ 이므로 $S_z \propto \text{Re}(k_z)|E|^2$, 따라서

$$T_s = |t|^2 \frac{\text{Re}(k_{zf})}{\text{Re}(k_{z0})} \quad (20)$$

p편광: $E_x \propto (k_z/\varepsilon)H_y$ 이고 $|H| = |\tilde{n}||E|$ 이므로

$$S_z \propto \text{Re}\left(\frac{k_z}{\tilde{n}^2}\right) |\tilde{n}|^2 |E|^2 = \frac{\text{Re}(\tilde{n}^2 k_z^*)}{|\tilde{n}|^2} |E|^2 \quad (21)$$

$$T_p = |t|^2 \frac{\text{Re}(\tilde{n}_f^2 k_{zf}^*)/|\tilde{n}_f|^2}{\text{Re}(\tilde{n}_0^2 k_{z0}^*)/|\tilde{n}_0|^2} \quad (22)$$

$\tilde{n}$ 가 실수면 두 식이 일치하지만 흡수 매질에서는 다릅니다. p에 s식을 쓰면 에너지 보존이 깨지고 오차는 각도와 $\text{Im}(\tilde{n})$ 에 따라 커집니다 (검증 A3에서 검출).

5.1 반무한 흡수 기판에서 T 의 의미

제가 실제로 틀렸던 지점입니다. 처음 검증 항목을 세울 때 “405 nm에서 Si는 불투명하니 $T \rightarrow 0$ ”을 기대했습니다. **테스트의 물리가 틀렸습니다.** 반무한 매질에는 두 번째 계면이 없으므로 T 는 기판 앞면을 통과해 들어간 플럭스입니다. 그 플럭스는 무한 경로에 걸쳐 흡수되지만 이미 들어갔습니다. 따라서 $R + T = 1$ 이 정확히 성립하고, 이 부기 체계에서 $A = 0$ 입니다. 불투명성은 **유한 슬랩**의 성질이며, 별도의 Beer-Lambert 검증(B13)으로 분리했습니다. 405 nm에서 $1/\alpha = 124.3 \text{ nm}$ 입니다.

6 물질 — 분산은 어디서 오는가

6.1 SiO_2 : Sellmeier는 감쇠 없는 Lorentz 진동자

$$\varepsilon(\omega) = 1 + \sum_j \frac{f_j \omega_{pj}^2}{\omega_j^2 - \omega^2 - i\gamma_j \omega} \quad (23)$$

투명 영역에서 $\gamma \rightarrow 0$ 으로 두고 $\omega = 2\pi c/\lambda$ 로 바꾸면

$$n^2 = 1 + \sum_j \frac{B_j \lambda^2}{\lambda^2 - C_j}, \quad B_j = \frac{f_j \omega_{pj}^2}{\omega_j^2}, \quad C_j = \lambda_j^2 \quad (24)$$

Malitson 계수의 물리적 정체는 다음과 같습니다.

$\sqrt{C_j}$	에너지	정체
68.4 nm	18.1 eV	심준위 전자 전이
116.2 nm	10.7 eV	SiO_2 밴드갭/엑시톤 (갭 $\sim 9 \text{ eV}$)
9.896 μm	125 meV (1010 cm^{-1})	Si-O-Si 신축 포논

즉 가시광 SiO_2 의 분산은 **UV 전자 전이와 IR 포논 사이에 끼인 꼬리**입니다. 세 항은 임의로 맞춘 피팅 파라미터가 아닙니다.

6.2 Si: 왜 Sellmeier가 안 되는가

Si는 가시·UV에서 밴드갭 흡수가 강해 $\gamma \rightarrow 0$ 가정이 붕괴합니다. 저차 해석식이 존재하지 않으므로 측정 표를 보간해야 합니다. 임계점 구조는 다음과 같습니다.

- $E_1 \approx 3.4$ eV (365 nm): Λ 방향 전이 $\rightarrow n$ 이 6.03까지 상승
- $E_2 \approx 4.25$ eV (292 nm): X/Σ 근처, ε_2 최대
- 간접갭 1.12 eV (1100 nm): 포논 보조 필요

k 를 로그 보간한 이유. Si의 k 는 250 nm에서 3.67, 1100 nm에서 3×10^{-5} — **5자리수**에 걸칩니다. 간접갭이라 밴드 끝 근처 흡수가 포논 보조로만 일어나 지수적으로 죽기 때문입니다. 지수 꼬리를 선형 보간하면 격자점 사이에서 크게 틀립니다. 따라서 n 은 선형, $\log k$ 는 선형 보간합니다.

7 역문제 — 왜 어려운가

7.1 근본 축퇴

투명막에서

$$R(d) \simeq A + B \cos(2\delta), \quad \delta = \frac{2\pi}{\lambda} n_f d \cos \theta_f \quad (25)$$

R 은 d 에 대해 주기적입니다. **서로 다른 두 축퇴**가 있으며, 해결책도 다릅니다.

축퇴	원인	해결
Flank (측면)	$\cos$ 이 우함수 $\rightarrow$ 한 주기 안에서 같은 R 이 두 번	위상 측정 (엘립소메트리)
Order (차수)	$\lambda/(2n_f \cos \theta_f)$ 마다 반복	광대역 또는 각도 스캔

실험 2의 설계 A(405 nm 단일점)에서 살아남은 후보 8개가 이 구조를 그대로 보여줍니다.

$$\underbrace{57.1, 79.5}_{\text{flank 쌍}} \quad \underbrace{194.9, 217.3}_{\text{flank 쌍}} \quad \underbrace{332.8, 355.1} \quad \underbrace{470.4, 492.9}$$

쌍 간 간격 137.82 nm는 예측값 $\lambda/(2n_f) = 137.79$ nm와 0.02% 일치합니다.

이것은 최적화 문제가 아닙니다. 단일파장·단일각 반사율 측정은 원리적으로 d 를 결정할 수 없고, 어떤 알고리즘·정칙화·평균화도 이를 고치지 못합니다. 정보가 데이터에 없습니다.

7.2 $n \cdot d$ 축퇴 — 반사계의 원죄

프린지 위상은 $n_f d$ 만 고정합니다. 따라서 가정한 굴절률이 틀리면

$$n_{\text{assumed}} d_{\text{fit}} = n_{\text{true}} d_{\text{true}} \implies \boxed{\frac{\Delta d}{d} = +\frac{\Delta n}{n}}, \quad \Delta n \equiv n_{\text{true}} - n_{\text{assumed}} \quad (26)$$

부호 방향에 주의. 가정 굴절률이 낮으면 적합 두께가 높게 나옵니다. 실험 4의 S3에서 $\Delta n = +0.002$ 에 대해 측정 편향 +0.3137 nm, 예측 +0.2984 nm로 5% 이내 일치를 확인했습니다. 한편 d 와 n 을 동시에 적합하면 상관계수는 $\rho = -0.930$ 으로 음수입니다 — 축퇴 골짜기를 따라 움직일 때 n 을 올리면 d 를 내려야 nd 가 유지되기 때문입니다. 두 부호는 서로 다른 상황을 기술하며 모순이 아닙니다.

ρ 가 정확히 -1 이 아닌 유일한 이유는 **분산**입니다. $n_{\text{SiO}_2}(\lambda)$ 는 파장에 따라 변하는데 상수 오프셋은 변하지 않으므로 스펙트럼 형태에 미세한 차이가 남습니다. 협대역 측정에서는 이 차이마저 사라져 n 과 d 를 동시에 결정할 수 없습니다.

8 정밀도의 한계 — Fisher 정보량과 Cramér-Rao 하한

8.1 유도

측정 R_i 가 독립 가우시안 잡음 σ 를 가지면

$$\ln L = -\frac{1}{2} \sum_i \frac{(R_i^{\text{meas}} - R_i(d))^2}{\sigma^2} + \text{const} \quad (27)$$

Fisher 정보량은

$$F_{ab} = \sum_i \frac{1}{\sigma_i^2} \frac{\partial R_i}{\partial p_a} \frac{\partial R_i}{\partial p_b}, \quad \text{Cov}(\hat{p}) \succeq F^{-1} \quad (28)$$

단일 파라미터에서는

$$\sigma_d \geq \frac{\sigma_R}{\sqrt{\sum_i (\partial R_i / \partial d)^2}} \quad (29)$$

8.2 물리적 읽기

$$\frac{\partial R}{\partial d} = -2B \cdot \frac{2\pi n_f \cos \theta_f}{\lambda} \sin 2\delta \quad (30)$$

세 가지가 즉시 보입니다.

1. **감도** $\propto 1/\lambda$ — 405 nm가 633 nm보다 유리합니다 (실험 1: 6.90 vs $3.90 \times 10^{-3}/\text{nm}$).
2. **감도** $\propto B$ (**프린지 대비**) — SiO_2/Si 는 $r_{01} \approx -0.19$, $r_{12} \approx -0.45$ 로 크기가 비슷해 대비가 좋습니다.
3. $\sin 2\delta = 0$ 인 **두께에서 감도 0** — 반주기(405 nm에서 68.9 nm)마다 **사각지대**가 존재합니다.

8.3 가장 중요한 비대칭

CRLB는 분산(랜덤 오차)만 제한합니다. 편향(계통 오차)에 대해서는 아무 말도 하지 않습니다. 적합 결과가 CRLB에 정확히 앉아 있으면서 동시에 1 nm 틀릴 수 있습니다. 실험 3(랜덤)과 실험 4(계통)를 끝까지 분리해 서술하는 이유가 이것입니다. 실측된 효율은 $\text{CRLB}/\sigma_{\text{MC}} = 1.019$ 로, 추정기가 데이터의 정보를 사실상 전부 뽑아냅니다 — 즉 **더 좋아지려면 알고리즘이 아니라 측정을 바꿔야 합니다.**

8.4 계통오차의 물리

계면층 (S1). 열산화 SiO_2 는 결정 Si와 급격히 만나지 않고, 아준화화량론 SiO_x 전이 영역이 존재합니다. 얇은 층은 r_{12} 의 크기와 위상을 모두 섭동하는데, 피팅 모델은 d 하나만 조절할 수 있으므로 위상 성분을 d 로 흡수합니다. 전달계수는 0.745 nm/nm 입니다. **가장 위험한 점은 잔차 RMS가 3×10^{-4} 에 머문다는 것입니다** — 좋은 χ^2 가 답을 보증하지 않습니다.

거칠기 (S2). 특징 크기 $\ll \lambda$ 이면 준정적 근사가 성립해 유효 매질로 대체할 수 있습니다. Bruggeman 자기무결 조건은

$$f_a \frac{\varepsilon_a - \varepsilon}{\varepsilon_a + 2\varepsilon} + f_b \frac{\varepsilon_b - \varepsilon}{\varepsilon_b + 2\varepsilon} = 0 \implies 2\varepsilon^2 - b\varepsilon - \varepsilon_a \varepsilon_b = 0, \quad b = (3f_a - 1)\varepsilon_a + (2 - 3f_a)\varepsilon_b \quad (31)$$

왜 Maxwell-Garnett이 아닌가. MG는 호스트/내포물 구분을 요구합니다. 50:50에서는 둘 중 무엇이 호스트인지 정할 수 없으므로 자기무결(self-consistent) Bruggeman이 맞습니다.

그리고 이걸 측정 오차가 아니라 정의 문제입니다. 측정된 전달계수 0.438 nm/nm 는 피팅이 물리적 표면 위치가 아니라 **부피등가 두께** $d + f t_{\text{rough}}$ 를 보고한다는 뜻입니다. 거칠기 규약이 다른 두 장비는 둘 다 완벽히 작동해도 $\sim f t_{\text{rough}}$ 만큼 불일치합니다. 실무에서 장비 간 매칭이 안 되는 흔한 원인입니다.

참조 데이터 (S4). Green 2008과 Aspnes 1983은 둘 다 동료심사를 거쳤고 어느 쪽도 틀리지 않았습니다. 405 nm에서 $\Delta n = +0.034$, 633 nm에서 -0.009 입니다. 제거 불가능하며 **계통 불확도로 안고 가야** 합니다.

8.5 최종 예산

원인	편향 (nm)	랜덤 바닥 대비
S1 계면 SiO_x 1 nm	+0.745	168
S2 거칠기 2 nm (Bruggeman)	+0.876	197
S3 막 굴절률 $\Delta n = +0.004$	+0.627	141
S4 Si 참조데이터 (Green $\leftrightarrow$ Aspnes)	+0.166	37
S5 각도 오차 0.1°	-0.147	33
RSS 합성	1.33	299

$\underbrace{4 \text{ pm}}_{\text{알고리즘 바닥}}$ vs $\underbrace{1.33 \text{ nm}}_{\text{현실 정확도}}$ 비율 ≈ 300

개선 방향은 옵티마이저가 아니라 **시료 지식**입니다.

Part II — 주석 달린 소스

9 omf/core.py — 순방향 모델

이 모듈이 프로젝트의 심장입니다. 259줄 중 실제 계산은 약 60줄이고, 나머지는 규약과 근거의 문서화입니다. 물리적으로 되돌리기 어려운 결정(규약, 분지, 플렉스 정의)이 전부 여기에 모여 있으므로 그렇게 설계했습니다.

9.1 규약 선언과 분지 상수

```
"""[... docstring: see Part III]"""

from __future__ import annotations

import numpy as np

__all__ = [
    "kz_from_index",
    "fresnel_rt",
    "coh_tmm",
    "snell_angles",
]

# Branch-selection tolerance. Values of |Im(kz)| below this are treated as
# exactly zero (a lossless propagating wave), for which we instead require
# Re(kz) > 0.
_BRANCH_TOL = 1e-10
```

왜 규약을 파일 최상단에 못 박는가. $e^{-i\omega t}$, $\tilde{n} = n + ik$, $e^{+ik_z z}$ 세 선택은 독립이 아니라 하나가 정해지면 나머지가 강제됩니다 (§ 3.1). 문헌의 절반은 $e^{+i\omega t}$, $\tilde{n} = n - ik$ 를 쓰며 둘 다 옳지만, 한 계산 안에서 섞으면 반드시 틀립니다. 이 파일에서 규약은 여기서 한 번 선언되고 이후 어디서도 재론되지 않습니다.

$_BRANCH_TOL = 1e-10$ 은 “무손실 전파”를 판정하는 수치 임계값입니다. $\text{Im}(k_z)$ 가 정확히 0인 경우는 부동소수점에서 거의 나오지 않으므로, 이 쪽 안에 들어오면 무손실로 간주하고 대신 $\text{Re}(k_z) > 0$ 을 요구합니다.

9.2 분지 선택 — 이 코드의 유일한 물리적 강제

```
# -----
# 1. Transverse wavevector and branch selection
# -----
def _fix_branch(kz: np.ndarray) -> np.ndarray:
    """Force Im(kz) >= 0; for the marginal lossless case force Re(kz) > 0. [... docstring: see Part III]"""
    kz = np.asarray(kz, dtype=complex)
    bad = (kz.imag < -_BRANCH_TOL) | (
        (np.abs(kz.imag) <= _BRANCH_TOL) & (kz.real < 0.0)
    )
    return np.where(bad, -kz, kz)
```

이 8줄이 특수 케이스를 전부 없앱니다. $+z$ 진행파는 $e^{i\text{Re}(k_z)z}e^{-\text{Im}(k_z)z}$ 이므로 수동 매질에서 감쇠하려면 $\text{Im}(k_z) \geq 0$ 이어야 합니다. 반대 분지는 (i) 거리에 따라 증폭되는 비물리적 파를 기술평고, (ii) 두꺼운 흡수층에서 $e^{ik_z d}$ 가 오버플로를 일으켜 계산을 파괴합니다.

두 번째 조건 ($|\text{Im}| \leq \text{tol}$) & ($\text{Re} < 0$)의 역할. 무손실 매질에서는 $\text{Im}(k_z) = 0$ 이라 첫 조건이 아무것도 걸러내지 못합니다. 이때는 부호를 정할 다른 기준이 필요한데, 그것이 “순방향 = $+z$ ”라는 정의, 즉 $\text{Re}(k_z) > 0$ 입니다. 이 줄이 없으면 무손실 층에서 전·후향파가 뒤바뀌는 미묘한 버그가 생깁니다.

`np.where`로 쓴 이유는 `if`가 아니라 배열 전체에 대해 동시에 적용해야 하기 때문입니다. 이것이 뒤의 벡터화를 가능하게 합니다.

9.3 면내 파수 보존 = 스넬 법칙

```
def kz_from_index(n: np.ndarray, lam_vac: np.ndarray, kx: np.ndarray) -> np.ndarray:
    """Normal component of the wavevector. [... docstring: see Part III]"""
    n = np.asarray(n, dtype=complex)
    k0 = 2.0 * np.pi / np.asarray(lam_vac, dtype=float)
    kz2 = (n * k0[...], None) ** 2 - np.asarray(kx, dtype=complex)[..., None] ** 2
    return _fix_branch(np.sqrt(kz2))

def snell_angles(n: np.ndarray, lam_vac: np.ndarray, kx: np.ndarray) -> np.ndarray:
    """Complex propagation angles, provided for reporting only. [... docstring: see Part III]"""
    n = np.asarray(n, dtype=complex)
    k0 = 2.0 * np.pi / np.asarray(lam_vac, dtype=float)
    return np.arcsin(np.asarray(kx, dtype=complex)[..., None] / (n * k0[...], None))
```

k_x 를 보존량으로 쓴 것이 이 코드의 가장 중요한 설계 결정입니다. 면내 병진 대칭성으로부터 $k_x = n_0 k_0 \sin \theta_0$ 이 모든 층에서 같고, $k_{z,j} = \sqrt{(\tilde{n}_j k_0)^2 - k_x^2}$ 입니다. 이것이 스넬 법칙 그 자체입니다.

각도 $\theta_j = \arcsin(k_x / \tilde{n}_j k_0)$ 로 쓰면 흡수 매질에서 θ_j 가 복소수가 되고, 복소 $\arcsin$ 이 어느 리만 시트에 있는지를 매번 추적해야 합니다. k_z 로 쓰면 전반사·흡수·소멸파가 전부 “ k_z 가 복소수가 된다”는 한 문장으로 흡수됩니다. **if 분기가 0개인 코드**가 나온 이유입니다.

snell_angles가 존재하지만 슬버는 이를 한 번도 호출하지 않습니다. 보고용입니다 — 사람이 읽을 때는 각도가 직관적이기 때문입니다.

브로드캐스팅 세부. n은 (... , N), k0와 kx는 (...) 모양이므로 [... , None]로 축을 하나 늘려 층 축과 정렬시킵니다. 이 shape 규약이 모듈 전체를 관통합니다.

9.4 Fresnel 계수

```

108 # -----
109 # 2. Interface (Fresnel) coefficients
110 # -----
111 def fresnel_rt(pol: str, n_i, n_j, kz_i, kz_j):
112     """Amplitude reflection / transmission coefficients for interface i -> j. [... docstring: see Part III]"""
113     n_i = np.asarray(n_i, dtype=complex)
114     n_j = np.asarray(n_j, dtype=complex)
115
116     if pol == "s":
117         denom = kz_i + kz_j
118         r = (kz_i - kz_j) / denom
119         t = 2.0 * kz_i / denom
120     elif pol == "p":
121         denom = n_j**2 * kz_i + n_i**2 * kz_j
122         r = (n_j**2 * kz_i - n_i**2 * kz_j) / denom
123         t = 2.0 * n_i * n_j * kz_i / denom
124     else:
125         raise ValueError("pol must be 's' or 'p'")
126     return r, t

```

$\cos \theta$ 대신 k_z 로 쓴 효과. $\tilde{n}_m \cos \theta_m = k_{zm} / k_0$ 이므로 모든 비율에서 공통 인자 k_0 가 소거됩니다. 결과적으로 s편광은 $r = (k_{zi} - k_{zj}) / (k_{zi} + k_{zj})$ 라는 임피던스 정합 형태가 되고, 이는 전송선의 $\Gamma = (Z_2 - Z_1) / (Z_2 + Z_1)$ 과 글자 그대로 같습니다(§ 2.2).

p편광의 $\tilde{n}^2$ 인자는 $\mathbf{E}$ 가 아니라 $\mathbf{H}$ 가 접선 성분이기 때문에 나옵니다. $E_x \propto (k_z / \varepsilon) H_y$ 이므로 어드미턴스가 $Y_p \propto \tilde{n}^2 / k_z$ 로 뒤집힙니다. t_p 의 $2\tilde{n}_i \tilde{n}_j$ 는 H 진폭 결과를 E 진폭으로 환산한 것입니다.

r_p 의 부호는 규약입니다. 반사 후 $\mathbf{E}_p$ 의 양의 방향을 어떻게 정하느냐에 따라 달라지며, 이 코드는 Byrnes 규약(θ_B 아래에서 $r_p > 0$)을 씁니다. $|r_p|^2$ 는 규약 무관이므로 R_p 는 안전합니다. 다른 출처의 r_p 를 섞어 쓰면 사고가 납니다.

인자로 배열 슬라이스 $n[\dots, :-1]$ 와 $n[\dots, 1:]$ 를 받도록 설계해, **모든 계면쌍을 호출 한 번에** 처리합니다.

9.5 전달행렬 조립 — 함수의 골격

```

140 # -----
141 # 3. Coherent transfer matrix
142 # -----
143 def _matmul(a, b):
144     """Batched 2x2 matrix product over trailing axes."""
145     return np.einsum("...ij,...jk->...ik", a, b)
146
147 def coh_tmm(pol, n_list, d_list, th_0, lam_vac, want_fields=False):
148     """Fully coherent TMM for a stack of isotropic layers. [... docstring: see Part III]"""
149     d_list = np.asarray(d_list, dtype=float)
150     n_list = np.asarray(n_list, dtype=complex)
151     N = d_list.size
152
153     if n_list.shape[-1] != N:
154         raise ValueError(f"n_list last axis ({n_list.shape[-1]}) != len(d_list) ({N})")
155     if not (np.isinf(d_list[0]) and np.isinf(d_list[-1])):
156         raise ValueError("d_list[0] and d_list[-1] must be np.inf (semi-infinite media)")
157     if N >= 3 and not np.all(np.isfinite(d_list[1:-1])):
158         raise ValueError("interior thicknesses must be finite")
159
160     th_0 = np.asarray(th_0, dtype=float)
161     lam_vac = np.asarray(lam_vac, dtype=float)
162
163     # ---- broadcast batch shape -----
164     batch = np.broadcast_shapes(th_0.shape, lam_vac.shape, n_list.shape[:-1])
165     th_0 = np.broadcast_to(th_0, batch)

```

입력 검증이 물리 조건입니다. 바깥 두 층의 두께가 inf 여야 하는 것은 반무한 매질 가정이고, 입사 매질이 실수 굴절률이어야 하는 것은 R 과 T 가 플렉스 비로 정의되려면 입사파 자체가 감쇠하지 않아야 하기 때문입니다. 흡수하는 매질에서 “입사 플렉스”는 위치에 따라 달라져 정의되지 않습니다.

`np.broadcast_shapes(th_0.shape, lam_vac.shape, n_list.shape[:-1])`가 이 모듈의 벡터화 핵심입니다. 파장 배열 (M ,)과 각도 배열 (K ,)를 각각 ($M, 1$), ($1, K$)로 넣으면 batch가 (M, K)가 되어 **2차원 스캔이 호출 한 번에** 끝납니다. 실험 1의 각도 스캔과 실험 2의 설계 B가 이 성질을 씁니다.

9.6 위상과 계면 항 — 벡터화가 일어나는 지점

```
187 lam_vac = np.broadcast_to(lam_vac, batch)
188 n_list = np.broadcast_to(n_list, batch + (N,))
189
190 k0 = 2.0 * np.pi / lam_vac
191 n0 = n_list[..., 0]
192 if np.max(np.abs(n0.imag)) > 1e-12:
193     raise ValueError("the incident medium must be lossless (real n) for R/T to be well defined")
194
195 # In-plane wavevector: conserved through the whole stack.
196 kx = (n0.real * k0) * np.sin(th_0)
197
198 kz = kz_from_index(n_list, lam_vac, kx)          # (... , N)
199
200 # ---- propagation phases for interior layers -----
```

여기서 슬라이싱 두 줄이 전체 스택을 처리합니다. `delta`는 내부층만 ($\dots, N-2$), `r_if/t_if`는 계면 수만큼 ($\dots, N-1$)입니다. 층 인덱스가 배열 축으로 올라가 있기 때문에 파이썬 루프가 필요 없습니다.

$\text{Im}(\delta) \geq 0$ 이 보장되는 것이 안전성의 근거입니다. 분지 규칙 덕분에 $e^{+i\delta}$ 의 크기가 1을 넘지 않으므로, 두께가 아무리 커도 오버플로가 발생하지 않습니다. 분지 교정을 생략하면 정확히 여기서 계산이 터집니다.

9.7 총 행렬과 누적 곱

```
201 # exp(+i delta) never overflows.
202 delta = kz[..., 1:-1] * d_list[1:-1]          # (... , N-2)
203
204 # ---- interface coefficients -----
205 r_if, t_if = fresnel_rt(pol, n_list[..., :-1], n_list[..., 1:],
206                        kz[..., :-1], kz[..., 1:]) # (... , N-1)
207
208 # ---- assemble M = D_01 * (P_1 D_12) * (P_2 D_23) * ... -----
209 def D(idx):
210     """Interface matrix (1/t)[[1, r],[r, 1]] for interface `idx`."""
211     r = r_if[..., idx]
212     t = t_if[..., idx]
213     m = np.empty(batch + (2, 2), dtype=complex)
214     m[..., 0, 0] = 1.0
215     m[..., 0, 1] = r
216     m[..., 1, 0] = r
217     m[..., 1, 1] = 1.0
218     return m / t[..., None, None]
219
220 def P(j):
221     """Propagation matrix diag(exp(-i delta), exp(+i delta)) for film `j`."""
222     dl = delta[..., j]
223     m = np.zeros(batch + (2, 2), dtype=complex)
224     m[..., 0, 0] = np.exp(-1j * dl)
225     m[..., 1, 1] = np.exp(+1j * dl)
226     return m
227
228 M = D(0)
229 for j in range(N - 2):          # one propagation + one interface each
230     M = _matmul(M, P(j))
231     M = _matmul(M, D(j + 1))
232
233 # [E0+, E0-]^T = M [Ef+, 0]^T  ->  t = 1/M11,  r = M21/M11
234 r = M[..., 1, 0] / M[..., 0, 0]
```

루프가 데이터가 아니라 층을 돕니다. 이것이 196배 가속의 정체입니다. 층은 보통 3~5개인데 데이터 점은 수천 개이므로, 루프를 어디에 두느냐가 성능을 결정합니다. `einsum("...ij,...jk->...ik")`가 batch 차원 전체에 대해 2×2 곱을 한 번에 수행합니다.

D 와 P 의 물리적 역할이 분리되어 있습니다. D (계면)는 전·후향 진폭을 섞고, P (전파)는 위상만 축적합니다. Airy 급수를 무한히 더하는 대신 이 두 연산을 번갈아 곱하는 것이 전달행렬의 전부입니다 — 급수를 미리 합해둔 형태입니다 (§ 4.3).

$r = M_{21}/M_{11}$, $t = 1/M_{11}$ 의 근거. 경계조건 $[E_0^+, E_0^-]^T = M[E_f^+, 0]^T$ 에서 출사 매질에 후향파가 없다($E_f^- = 0$)는 조건을 쓰면 $E_0^+ = M_{11}E_f^+$, $E_0^- = M_{21}E_f^+$ 이므로 두 식이 즉시 따라옵니다. $N = 3$ 에 대해 직접 곱해보면 정확히 Airy 공식이 나오며, 검증 B5가 이를 2×10^{-16} 로 확인합니다.

9.8 반사율과 투과율 — p편광의 켈레

```

238 R = np.abs(r) ** 2
239
240 # ---- transmittance: needs the ratio of longitudinal Poynting flux -----
241 # s-pol : T = |t|^2 Re(n_f cos th_f) / Re(n_0 cos th_0)
242 # p-pol : T = |t|^2 Re(n_f cos th_f^*) / Re(n_0 cos th_0^*)
243 # The conjugation in the p case is NOT cosmetic: for an absorbing exit medium
244 # the two prefactors differ, and using the s-form for p violates energy
245 # conservation. Rewritten with n cos th = k_z / k_0:
246 # Re(n cos th) = Re(k_z)/k_0
247 # Re(n cos th^*) = Re(n^2 k_z^*) / (k_0 |n|^2)
248 nf, kzf = n_list[... , -1], kz[... , -1]
249 if pol == "s":
250     num, den = kzf.real, kz[... , 0].real
251 else:
252     num = (nf**2 * np.conj(kzf)).real / np.abs(nf) ** 2
253     den = (n0**2 * np.conj(kz[... , 0])).real / np.abs(n0) ** 2
254 T = np.abs(t) ** 2 * (num / den)
255
256 out = {"r": r, "t": t, "R": R, "T": T, "A": 1.0 - R - T, "kz": kz, "kx": kx}
257 if want_fields:
258     out["M"] = M
259 return out

```

T 는 $|t|^2$ 가 아닙니다. z 방향 시간평균 Poynting 플럭스의 비입니다. s 편광은 $S_z \propto \text{Re}(k_z)|E|^2$ 이라 인자가 $\text{Re}(k_{zf})/\text{Re}(k_{z0})$ 이지만, p 편광은 $S_z \propto \text{Re}(\tilde{n}^2 k_z^*)|E|^2/|\tilde{n}|^2$ 이라 **켈레의 위치가 다릅니다**.

$\tilde{n}$ 가 실수면 두 식이 일치하므로 무슨실 문제만 다루면 이 차이를 영영 발견하지 못합니다. 흡수 매질에서 p 에 s 식을 쓰면 에너지 보존이 깨지고, 오차는 각도와 $\text{Im}(\tilde{n})$ 에 따라 커집니다. 검증 A3가 이를 잡아냅니다.

$A = 1 - R - T$ 의 의미에 주의. 반무한 흡수 기판에서는 T 가 “기판 안으로 들어간” 플럭스이므로 $R + T = 1$ 이고 $A = 0$ 이 됩니다. 기판 내부의 흡수는 A 가 아니라 T 에 계산됩니다. 불투명성은 **유한 슬랩**의 성질입니다(§ 5.1, 검증 B12/B13).

10 omf/materials.py — 굴절을 공급

두 종류의 모델이 공존하며, 그 이유가 물리적으로 명확합니다. 투명 영역에서는 해석식 (Sellmeier)이 정확하고, 강한 밴드간 흡수가 있으면 해석식이 존재하지 않아 측정 표를 보간해야 합니다.

10.1 Sellmeier — 감쇠 없는 Lorentz 진동자

```

38 # -----
39 # Sellmeier models
40 # -----
41 def _sellmeier(lam_nm, B, C):
42     """n^2 = 1 + sum_j B_j lam^2/(lam^2 - C_j), with lam in micrometres."""
43     lam_um2 = (np.asarray(lam_nm, dtype=float) / 1000.0) ** 2
44     n2 = 1.0
45     for b, c in zip(B, C):
46         n2 = n2 + b * lam_um2 / (lam_um2 - c)
47     return np.sqrt(n2).astype(complex)
48
49 def n_sio2(lam_nm):
50     """Fused silica, 20 C. Malitson, J. Opt. Soc. Am. 55, 1205 (1965). [... docstring: see Part III]"""
51     B = (0.6961663, 0.4079426, 0.8974794)
52     C = (0.0684043**2, 0.1162414**2, 9.896161**2)
53     return _sellmeier(lam_nm, B, C)

```

Sellmeier는 피팅 공식이 아니라 물리 모델입니다. Lorentz 진동자 합 $\epsilon = 1 + \sum_j f_j \omega_{pj}^2 / (\omega_j^2 - \omega^2 - i\gamma_j \omega)$ 에서 투명 영역($\gamma \rightarrow 0$) 극한을 취하고 $\omega \rightarrow \lambda$ 로 바꾼 것이 정확히 $n^2 = 1 + \sum_j B_j \lambda^2 / (\lambda^2 - C_j)$ 입니다.

Malitson 계수의 $\sqrt{C_j}$ 는 각각 68.4 nm(18.1 eV, 심준위), 116.2 nm(10.7 eV, SiO₂ 밴드갭/엑시톤), 9.896 μ m(125 meV, Si-O-Si 신축 포논)입니다. **가시광 SiO₂의 분산은 UV 전자 전이와 IR 포논 사이에 끼인 꼬리입니다.**

$k = 0$ 은 구성상 그렇게 되는 것이지 근사가 아닙니다 — $\gamma \rightarrow 0$ 을 이미 가정했기 때문입니다. 용융실리카의 가시광 흡수는 $\alpha < 10^{-4}$ /cm이라 이 가정이 훌륭합니다.

docstring에 적힌 경고가 실험 4의 S3로 이어집니다. Si 위에 성장한 열산화 SiO₂는 벌크 용융실리카와 같지 않습니다. 치밀화와 응력으로 가시광 n 이 보통 0.002~0.006 높습니다. 이를 무시하면 217 nm 막에서 0.6 nm 편향이 생깁니다.

10.2 비분산 매질과 상수 공장

```

48 def n_bk7(lam_nm):
49     """N-BK7 borosilicate crown (SCHOTT). Valid 300 nm - 2.5 um. [... docstring: see Part III]"""
60     B = (1.03961212, 0.231792344, 1.01046945)
61     C = (0.00600069867, 0.0200179144, 103.560653)
62     return _sellmeier(lam_nm, B, C)
63
64
65
66
67 def n_air(lam_nm):
68     """Air at STP, treated as non-dispersive. [... docstring: see Part III]"""
69     return np.full(np.shape(lam_nm), 1.000293, dtype=complex)
70
71
72
73 def n_vacuum(lam_nm):
74     return np.full(np.shape(lam_nm), 1.0, dtype=complex)
75
76
77
78 def n_constant(value):
79     """Factory for a dispersionless material of given complex index."""
80     def f(lam_nm):
81         return np.full(np.shape(lam_nm), complex(value), dtype=complex)
82     return f

```

`n_air`가 분산을 무시하는 근거는 정량적입니다 — 가시광 전역에서 공기의 분산은 $\sim 3 \times 10^{-6}$ 으로, 이 프로젝트가 분해할 수 있는 어떤 효과보다도 작습니다. 근사를 쓸 때는 “작다”가 아니라 **무엇보다 작은지**를 밝혀야 합니다.

`n_bk7`의 docstring은 실제 커버슬립이 D263이나 B270이라 n_d 가 1.5230 대 1.5168로 약 0.4% 다르다고 명시합니다. Brewster 각 시연에는 충분하지만 절대 두께 계측에는 부족합니다 — 이런 구분을 코드 안에 남겨두는 것이 나중에 자신을 속이지 않는 방법입니다.

`n_constant`는 클로저를 반환하는 공장 함수입니다. 실험 4에서 계면층 $n = 2.5$ 를 끼워 넣을 때 쓰이며, 덕분에 “물질”을 이름·함수·상수 어느 것으로도 지정할 수 있는 통일 인터페이스가 됩니다.

10.3 Si — 표 보간과 로그 스케일

```

83 # -----
84 # Tabulated silicon
85 # -----
86 def _load_si_table(fname="Si_Green2008.csv"):
87     path = os.path.join(_DATA_DIR, fname)
88     rows = []
89     with open(path) as fh:
90         for line in fh:
91             line = line.strip()
92             if not line or line.startswith("#") or line[0].isalpha():
93                 continue
94             # skip provenance comments and the column header
95             rows.append([float(x) for x in line.split(",")])
96     arr = np.asarray(rows, dtype=float)
97     lam, n, k = arr[:, 0], arr[:, 1], arr[:, 2]
98     order = np.argsort(lam)
99     return lam[order], n[order], k[order]
100
101
102
103 _SI_LAM, _SI_N, _SI_K = _load_si_table()
104 SI_RANGE_NM = (float(_SI_LAM[0]), float(_SI_LAM[-1]))
105
106
107 def n_si(lam_nm):
108     """Intrinsic crystalline silicon at 300 K. [... docstring: see Part III]"""
109     lam = np.asarray(lam_nm, dtype=float)
110     if np.any(lam < _SI_LAM[0] - 1e-9) or np.any(lam > _SI_LAM[-1] + 1e-9):
111         raise ValueError(
112             f"n_si: wavelength outside tabulated range {_SI_RANGE_NM} nm"
113         )
114     n = np.interp(lam, _SI_LAM, _SI_N)
115     floor = 1e-12
116     logk = np.interp(lam, _SI_LAM, np.log(np.maximum(_SI_K, floor)))
117     k = np.exp(logk)
118     return (n + 1j * k).astype(complex)

```

왜 Si에는 해석식이 없는가. 가시·UV에서 밴드갭 흡수가 강해 $\gamma \rightarrow 0$ 가정이 붕괴합니다. 임계점은 $E_1 \approx 3.4$ eV(365 nm, Λ 방향)와 $E_2 \approx 4.25$ eV(292 nm, X/Σ 근처)이며, E_1 에서 n 이 6.03까지 치솟습니다. 저차 다항식이나 Sellmeier로는 이 구조를 재현할 수 없습니다.

로그 k 보간이 핵심입니다. k 는 250 nm의 3.67에서 1100 nm의 3×10^{-5} 까지 5자리수에 걸칩니다. Si의 갭이 간접갭이라 밴드 끝 근처 흡수는 포논 보조로만 일어나 지수적으로 죽기 때문입니다. 지수 꼬리를 선형 보간하면 격자점 사이에서 크게 틀립니다. n 은 완만하므로 선형으로 충분합니다.

범위 밖 예외를 던지는 이유. `np.interp`는 범위 밖에서 조용히 끝점 값을 반환합니다. 계측 코드에서 이는 최악의 실패 모드입니다 — 틀린 답이 경고 없이 나옵니다. 명시적으로 예외를 던져 사용자가 알아채도록 강제합니다.

10.4 두 번째 Si 데이터셋 — 계통오차를 재기 위한 장치

```

140 _ASP_LAM, _ASP_N, _ASP_K = _load_si_table("Si_Aspnes1983.csv")
141
142 def n_si_aspnes(lam_nm):
143     """Crystalline silicon, D. E. Aspnes & A. A. Studna, PRB 27, 985 (1983). [... docstring: see Part III]"""
144     lam = np.asarray(lam_nm, dtype=float)
145     if np.any(lam < _ASP_LAM[0] - 1e-9) or np.any(lam > _ASP_LAM[-1] + 1e-9):
146         raise ValueError(
147             f"n_si_aspnes: outside tabulated range "
148             f"({_ASP_LAM[0]}, {_ASP_LAM[-1]}) nm")
149     n = np.interp(lam, _ASP_LAM, _ASP_N)
150     k = np.exp(np.interp(lam, _ASP_LAM, np.log(np.maximum(_ASP_K, 1e-12))))
151     return (n + 1j * k).astype(complex)
152
153 MATERIALS = {
154     "air": n_air,
155     "vacuum": n_vacuum,
156     "SiO2": n_sio2,
157     "BK7": n_bk7,
158     "Si": n_si,
159     "Si_Aspnes": n_si_aspnes,
160 }
161
162 def build_n_list(names_or_values, lam_nm):
163     """Assemble an (... , N) index array from a list of material names/values. [... docstring: see Part III]"""
164     lam = np.asarray(lam_nm, dtype=float)
165     cols = []
166     for item in names_or_values:
167         if isinstance(item, str):
168             cols.append(np.broadcast_to(MATERIALS[item](lam), lam.shape))
169         elif callable(item):
170             cols.append(np.broadcast_to(item(lam), lam.shape))
171         else:
172             cols.append(np.full(lam.shape, complex(item), dtype=complex))
173     return np.stack(cols, axis=-1)

```

Aspnes 데이터는 편의 기능이 아니라 실험 장비입니다. Green 2008과 Aspnes 1983은 둘 다 동료심사를 거쳤고 어느 쪽도 틀리지 않았습니다. 405 nm에서 $\Delta n = +0.034$, 633 nm에서 -0.009 차이가 납니다.

이 차이는 수치 잡음이 아니라 **실재하는 참조 데이터 불일치**이며, 독립 측정 없이는 제거할 수 없습니다. 두 데이터셋을 모두 탑재해 한쪽으로 생성하고 다른 쪽으로 적합하면 그 불일치가 두께 편향 몇 nm에 해당하는지 정량화됩니다(실험 4의 S4: 0.166 nm). **제거하는 대신 예산에 실어 나르는 것이** 정직한 처리입니다.

build_n_list는 이름·콜러블·복소상수를 모두 받아 (... , N) 배열로 쌓습니다. 이 통일 인터페이스 덕분에 실험 4에서 층 하나를 끼워 넣거나 굴절률에 오프셋을 더하는 조작이 한 줄로 끝납니다.

11 omf/fitter.py — 역문제

순방향 모델이 정확해도 역문제는 별개의 어려움을 갖습니다. 이 모듈의 존재 이유는 $R(d)$ 가 d 에 대해 주기적이라는 사실 하나입니다.

11.1 모듈 서두 — 왜 단순 최소제곱이 실패하는가

```

"""
omf.fitter -- the inverse problem: recover film thickness (and optionally index)
from measured reflectance.

Why this is not a trivial least-squares problem
-----
For a transparent film on a substrate the reflectance is, to leading order,

    R(d) ~ A + B cos(2 delta),    delta = (2 pi / lam) n_f d cos(theta_f)

i.e. **periodic in d**. A film that is thicker by one full order,

    Delta d = lam / (2 n_f cos theta_f)  (~139 nm for SiO2 at 405 nm, normal incidence)

produces an almost identical single-wavelength reflectance. The cost function
therefore has a comb of near-degenerate local minima, and a gradient-based
optimiser started at an arbitrary guess will simply fall into whichever tooth of
the comb it happens to be nearest. This is the classic **fringe-order
ambiguity** of thin-film metrology.

Two things break the degeneracy, and the code exposes both:

1. **Coarse grid pre-scan.** Evaluate the cost on a dense grid spanning the

```

```

24 plausible thickness range, take the global minimum as the starting point,
25 then refine locally. Cheap because the forward model is vectorised.
26 2. **Spectroscopic / angle-resolved data.** The fringe spacing itself is
27 wavelength-dependent ( $\Delta \propto 1/\lambda$ ), so a broadband measurement pins the
28 absolute order. A single-wavelength, single-angle measurement genuinely
29 cannot -- no algorithm fixes that; it is an information-content limit, not an
30 optimiser limit.
31 """
32
33 from __future__ import annotations
34
35 from dataclasses import dataclass, field
36 from typing import Callable, Sequence
37
38 import numpy as np
39 from scipy.optimize import least_squares
40
41 from .core import coh_tmm
42 from .materials import build_n_list
43
44 __all__ = ["FilmModel", "FitResult"]

```

프린지 차수 측되는 알고리즘 결함이 아니라 정보의 부재입니다. $R \simeq A + B \cos 2\delta$ 이므로 $\Delta d = \lambda / (2n_f \cos \theta_f)$ 만큼 두꺼운 막은 단일 파장에서 거의 같은 반사율을 냅니다(405 nm에서 137.79 nm). 비유함수가 빗살 모양이 되고, 경사법은 시작점에서 가장 가까운 빗살 이빨로 떨어집니다.

서두에서 두 해결책을 명확히 구분합니다. (1) 조밀 격자 사전탐색은 오퍼마이저 문제를 풀고, (2) 광대역/각도 데이터는 정보 문제를 푹니다. 후자가 없으면 전자로도 못 고칩니다 — 실험 2의 설계 A가 이를 실증합니다.

11.2 FitResult — 불확도를 결과에 동봉

```

43 @dataclass
44 class FitResult:
45     """Outcome of a fit, with uncertainty propagated from the Jacobian."""
46     params: np.ndarray          # best-fit parameter vector
47     names: list                 # parameter names
48     sigma: np.ndarray           # 1-sigma from the covariance diagonal
49     cov: np.ndarray             # parameter covariance matrix
50     rms: float                  # RMS of the residual
51     coarse_best: np.ndarray     # starting point from the grid pre-scan
52     success: bool = True
53     nfev: int = 0
54     extra: dict = field(default_factory=dict)
55
56     def __repr__(self):
57         lines = [f"FitResult(rms={self.rms:.3e}, nfev={self.nfev})"]
58         for nm, p, s in zip(self.names, self.params, self.sigma):
59             lines.append(f"    {nm:>10s} = {p:12.5f} +/- {s:.5f}")
60         return "\n".join(lines)
61
62     @property
63     def correlation(self):
64         d = np.sqrt(np.diag(self.cov))
65         outer = np.outer(d, d)
66         with np.errstate(invalid="ignore", divide="ignore"):
67             return np.where(outer > 0, self.cov / outer, np.nan)

```

추정값과 불확도를 한 객체로 묶은 것이 설계 의도입니다. 계측에서 불확도 없는 숫자는 정보가 아닙니다. `correlation` 속성이 별도로 있는 이유는 d - n 상관계수가 이 문제의 핵심 진단량이기 때문입니다(실험 3에서 $\rho = -0.930$).

상관행렬을 계산할 때 `errstate`로 0-나눗셈을 막고 `nan`으로 두는 것은, 특이한 공분산(정보가 없는 방향)을 조용히 0으로 만들지 않기 위해서입니다.

11.3 FilmModel 생성자 — 무엇을 자유 파라미터로 둘 것인가

```

75 class FilmModel:
76     """A stack with one free-thickness film and an optional index offset. [... docstring: see Part III]"""
77
78     def __init__(self, materials: Sequence, d_nm: Sequence[float],
79                  fit_layer: int = 1, fit_dn: bool = False, pol: str = "s"):
80         self.materials = list(materials)
81         self.d_nm = np.asarray(d_nm, dtype=float)
82         self.fit_layer = fit_layer
83         self.fit_dn = fit_dn
84         self.pol = pol
85
86         if pol not in ("s", "p", "unpol"):
87             raise ValueError("pol must be 's', 'p' or 'unpol'")
88         if not (np.isinf(self.d_nm[0]) and np.isinf(self.d_nm[-1])):
89             raise ValueError("outer layers must have infinite thickness")

```

```

88     if not np.isfinite(self.d_nm[fit_layer]):
89         raise ValueError("fit_layer must be a finite-thickness film")
90
91     self.names = ["d_nm"] + (["dn"] if fit_dn else [])
92
93     # -----
94     # forward model
95     # -----

```

fit_dn 옵션이 이 클래스의 물리적 긴장 지점입니다. 열산화막과 벌크 용융실리카의 굴절률 차이를 흡수하려면 n 을 자유롭게 두어야 하지만, **프린지가 실제로 재는 것은 광학 두께 $n \cdot d$ 이므로 d 와 n 이 강하게 반상관됩니다.**

docstring이 이 함정을 미리 경고합니다. 실측된 결과는 $\rho = -0.930$ 이고 σ_d 가 2.7배 악화됩니다. ρ 가 정확히 -1 이 아닌 유일한 이유는 분산입니다 — $n_{\text{SiO}_2}(\lambda)$ 는 파장에 따라 변하는데 상수 오프셋은 변하지 않으므로 스펙트럼 형태에 미세한 차이가 남습니다. **현대역 측정에서는 이 차이마저 사라져 n 과 d 를 동시에 결정할 수 없습니다.**

생성자 검증(바깥 층 inf, 적합 대상 층은 유한)은 물리적으로 말이 되는 스택만 허용하기 위한 것입니다.

11.4 순방향 래퍼

```

110 def reflectance(self, params, lam_nm, theta_rad):
111     """R(lam, theta) for the given parameter vector. [... docstring: see Part III]"""
112     params = np.atleast_1d(np.asarray(params, dtype=float))
113     d = self.d_nm.copy()
114     d[self.fit_layer] = params[0]
115
116     lam = np.asarray(lam_nm, dtype=float)
117     n_list = build_n_list(self.materials, lam)
118
119     if self.fit_dn:
120         n_list = n_list.copy()
121         n_list[... , self.fit_layer] += params[1]
122
123     if self.pol == "unpol":
124         rs = coh_tmm("s", n_list, d, theta_rad, lam)["R"]
125         rp = coh_tmm("p", n_list, d, theta_rad, lam)["R"]
126         return 0.5 * (rs + rp)
127     return coh_tmm(self.pol, n_list, d, theta_rad, lam)["R"]
128
129 # -----
130 # coarse grid pre-scan
131 # -----

```

'unpol'은 R_s 와 R_p 의 **강도 평균**입니다. 진폭 평균이 아닙니다 — 두 편광은 서로 코히런트하지 않으므로 간섭하지 않고, 검출기가 보는 것은 전력의 합입니다. 이를 진폭에서 평균하면 존재하지 않는 간섭항이 생깁니다.

n_list 를 매 호출마다 build_n_list로 재생성하는 것은 비효율처럼 보이지만, 분산 모델이 파장에만 의존하므로 캐싱하면 fit_dn 경로에서 오염이 생깁니다. .copy() 후 오프셋을 더하는 것도 같은 이유입니다.

11.5 조밀 격자 사전탐색 — 전역성 확보

```

140 def coarse_scan(self, lam_nm, theta_rad, R_meas, d_grid, dn_grid=None):
141     """Evaluate SSR on a grid and return the global minimiser. [... docstring: see Part III]"""
142     d_grid = np.asarray(d_grid, dtype=float)
143     dn_grid = np.array([0.0]) if dn_grid is None else np.asarray(dn_grid, float)
144     R_meas = np.asarray(R_meas, dtype=float)
145
146     ssr = np.empty((d_grid.size, dn_grid.size))
147     for i, d in enumerate(d_grid):
148         for j, dn in enumerate(dn_grid):
149             p = (d, dn) if self.fit_dn else (d,)
150             resid = self.reflectance(p, lam_nm, theta_rad) - R_meas
151             ssr[i, j] = float(np.sum(resid**2))
152
153     i, j = np.unravel_index(np.argmin(ssr), ssr.shape)
154     best = [d_grid[i]] + ([dn_grid[j]] if self.fit_dn else [])
155     return np.array(best), ssr, d_grid, dn_grid
156
157 # -----
158 # full fit
159 # -----

```

이 무식한 전수 탐색이 가능한 이유가 벡터화입니다. 격자점 하나마다 reflectance를 호출하지만, 그 호출 자체가 이미 모든 파장에 대해 벡터화되어 있습니다. 1200점 격자 $\times$ 251 파장이면 30만 회 TMM 평가인데 1초 미만입니다. core.py에서 루프를 층으로 옮긴 결정이 여기서 배당을 지급합니다.

실측된 효과. 소박한 국소 최적화는 9개 시작점 중 **6개**에서 틀린 프린지 차수로 수렴했습니다(47.0, 388.2, 506.2 nm 등).

사전탐색을 붙이면 9개 전부 성공합니다.

격자 간격은 프린지 주기보다 충분히 촘촘해야 합니다 — 137.79 nm 주기에 대해 1200점/600 nm는 0.5 nm 간격이므로 안전합니다.

11.6 정밀화와 공분산

```

165 def fit(self, lam_nm, theta_rad, R_meas, d_grid,
166         dn_grid=None, sigma=None, bounds=None, verbose=False):
167     """Coarse pre-scan followed by Levenberg-Marquardt-style refinement. [... docstring: see Part III]"""
168     R_meas = np.asarray(R_meas, dtype=float)
169     w = 1.0 if sigma is None else 1.0 / np.asarray(sigma, dtype=float)
170
171     p0, ssr_grid, dg, dng = self.coarse_scan(
172         lam_nm, theta_rad, R_meas, d_grid, dn_grid)
173
174     def resid(p):
175         return (self.reflectance(p, lam_nm, theta_rad) - R_meas) * w
176
177     if bounds is None:
178         lo = [float(np.min(d_grid))] + ([-0.5] if self.fit_dn else [])
179         hi = [float(np.max(d_grid))] + ([+0.5] if self.fit_dn else [])
180         bounds = (lo, hi)
181
182     res = least_squares(resid, p0, bounds=bounds, method="trf",
183                        xtol=1e-14, ftol=1e-14, gtol=1e-14,
184                        verbose=2 if verbose else 0)
185
186     m, npar = res.fun.size, res.x.size
187     J = res.jac
188     JTJ = J.T @ J
189     try:
190         cov = np.linalg.inv(JTJ)
191     except np.linalg.LinAlgError:
192         cov = np.full((npar, npar), np.nan)
193
194     if sigma is None:
195         dof = max(m - npar, 1)
196         chi2_red = float(np.sum(res.fun**2)) / dof
197         cov = cov * chi2_red
198     # if sigma was supplied the residuals are already whitened, so
199     # inv(J^T J) is directly the covariance.
200
201     rms = float(np.sqrt(np.mean(
202         (self.reflectance(res.x, lam_nm, theta_rad) - R_meas) ** 2)))
203
204     return FitResult(
205         params=res.x, names=self.names,
206         sigma=np.sqrt(np.abs(np.diag(cov))),
207         cov=cov, rms=rms, coarse_best=p0,
208         success=bool(res.success), nfev=int(res.nfev),
209         extra={"ssr_grid": ssr_grid, "d_grid": dg, "dn_grid": dng},
210     )

```

공분산 추정의 두 갈래가 통계적으로 다른 주장입니다.

- σ 를 주면 잔차가 이미 백색화되어 있으므로 $(J^T J)^{-1}$ 이 곧 공분산입니다. 이는 사전 추정 — “내 장비의 잡음이 이만큼임을 독립적으로 안다”.

- $\sigma=None$ 이면 축소 카이제곱 $\chi^2_{\text{red}} = \text{SSR}/(m - p)$ 을 곱합니다. 이는 사후 추정 — “잔차 크기로부터 잡음을 역산한다”. 모델이 틀렸을 때 χ^2_{red} 가 커져 불확도가 부풀려지므로, 어느 정도 자기 보호 기능이 있습니다.

그러나 이 보호는 불완전합니다. 실험 4의 S1에서 1 nm 계면층은 0.745 nm 편향을 만들지만 잔차 RMS는 3×10^{-4} 에 머물렀습니다. 좋은 χ^2 가 답을 보증하지 않습니다. 계통오차는 잔차에 거의 흔적을 남기지 않고 파라미터로 흡수되기 때문입니다.

xtol/ftol/gtol을 10^{-14} 로 조인 것은 실험 3에서 CRLB와 비교하기 위함입니다. 수렴 허용오차가 험거우면 몬테카를로 산포에 옵티마이저 잡음이 섞여 효율이 1보다 작게 나옵니다 — 측정하려는 양이 오염됩니다.

12 omf/uncertainty.py — 판정

이 모듈은 새로운 숫자를 만들지 않습니다. 앞의 모듈들이 만든 숫자가 **믿을 만한지**를 판정합니다. 계측 프로젝트와 단순 시뮬레이션을 가르는 지점입니다.

12.1 모듈 서두 — 왜 CRLB인가

```

"""
omf.uncertainty -- how well "can" a thickness be determined, and how well "is" it?

```

```

1 The distinction matters, and it is the part of a metrology project that a
2 fitting routine alone does not answer.
3
4 Cramer-Rao lower bound (CRLB)
5 -----
6 For measurements  $R_i$  with independent Gaussian noise of standard deviation
7  $\sigma_i$ , and a model  $R_i(p)$  depending on parameters  $p$ , the Fisher information
8 matrix is
9
10  $F_{ab} = \sum_i (1/\sigma_i^2) (dR_i/dp_a)(dR_i/dp_b)$ 
11
12 and *any* unbiased estimator obeys  $\text{Cov}(p) \succeq F^{-1}$  (matrix inequality). So
13
14  $\sigma_d \succeq \sqrt{(F^{-1})_{dd}}$ .
15
16 This is a statement about the *experiment*, not the algorithm: it says how much
17 information about  $d$  is physically present in the data. If the measured spread
18 sits at the CRLB, the estimator is efficient and the only way to do better is to
19 change the measurement (more points, lower noise, wider spectral range, a
20 different angle). If the measured spread is far above it, the estimator is
21 wasteful.
22
23 Note the crucial asymmetry: the CRLB bounds the *variance*, i.e. the random
24 error. It says nothing at all about *bias* from a wrong model. A fit can sit
25 exactly at the CRLB and still be 3 nm wrong because the true sample has a native
26 oxide the model omits. Random precision and systematic accuracy are separate
27 budgets (see scripts/exp04).
28 """
29
30 from __future__ import annotations
31
32 import numpy as np
33
34 __all__ = ["numerical_jacobian", "fisher_information", "crlb", "monte_carlo_fit"]

```

두 질문이 다릅니다. (1) 이 측정 설계에 정보가 얼마나 들어 있는가(CRLB) — 실험의 성질입니다. (2) 내 추정기가 그것을 다 뽑아내는가(몬테카를로) — 코드의 성질입니다. 둘을 비교해야 어디를 고쳐야 하는지 알 수 있습니다.

서두 마지막 문단이 이 프로젝트에서 가장 중요한 문장입니다. **CRLB는 분산만 제한하고 편향에 대해서는 아무 말도 하지 않습니다.** 적합 결과가 CRLB에 정확히 앉아 있으면서 동시에 3 nm 틀릴 수 있습니다 — 시료에 모델이 빠뜨린 자연 산화막이 있으면 그렇습니다. 랜덤 정밀도와 계통 정확도는 별개의 예산입니다.

12.2 수치 야코비안과 Fisher 정보량

```

40 def numerical_jacobian(f, p, steps):
41     """Central-difference Jacobian  $dF_i/dp_a$  of a vector-valued model. [... docstring: see Part III]"""
42     p = np.asarray(p, dtype=float)
43     steps = np.atleast_1d(np.asarray(steps, dtype=float))
44     cols = []
45     for a in range(p.size):
46         h = steps[a]
47         pp, pm = p.copy(), p.copy()
48         pp[a] += h
49         pm[a] -= h
50         cols.append((np.asarray(f(pp)) - np.asarray(f(pm))) / (2.0 * h))
51     return np.stack(cols, axis=-1) # (M, npar)
52
53 def fisher_information(J, sigma):
54     """ $F = J^T W J$  with  $W = \text{diag}(1/\sigma^2)$ ."""
55     sigma = np.broadcast_to(np.asarray(sigma, dtype=float), (J.shape[0],))
56     Jw = J / sigma[:, None]
57     return Jw.T @ Jw
58
59 def crlb(J, sigma):
60     """Return (sigma_lower_bound_vector, inverse Fisher matrix)."""
61     F = fisher_information(J, sigma)
62     Finv = np.linalg.pinv(F)
63     return np.sqrt(np.abs(np.diag(Finv))), Finv

```

중심차분을 쓴 이유. 절단오차가 $O(h^2)$ 이라 전방차분의 $O(h)$ 보다 한 자릿수 유리합니다. h 를 파라미터 스케일의 $\sim 10^{-4}$ 로 잡으면 절단오차가 고려하는 잡음 수준보다 충분히 아래로 내려갑니다. 두께에 대해 `steps=[0.02] nm`를 쓴 것이 이 판단입니다 — 프린지 주기 137.79 nm의 1.5×10^{-4} 입니다.

Fisher 행렬의 물리적 읽기. $F = J^T W J$ 에서 $\partial R / \partial d = -2B \cdot (2\pi n_f \cos \theta_f / \lambda) \sin 2\delta$ 이므로 세 가지가 보입니다: 감도 $\propto 1/\lambda$ (405 nm가 633 nm보다 유리), 감도 $\propto B$ (프린지 대비), 그리고 $\sin 2\delta = 0$ 인 두께마다 **감도가 정확히 0인 사각지대**가 반주기 간격으로 존재합니다.

F 의 역행렬 대각선이 각 파라미터의 하한 분산입니다. 다변수에서 $(F^{-1})_{dd}$ 가 $1/F_{dd}$ 보다 큰 것이 파라미터 상관의 대가입니다.

12.3 몬테카를로 — 그리고 inverse crime의 명시

```
def monte_carlo_fit(model, lam_nm, theta_rad, p_true, sigma, n_trials,
                    d_grid, dn_grid=None, rng=None, use_known_sigma=True):
    """Synthesise noisy data `n_trials` times and refit; return the estimates. [... docstring: see Part III]"""
    rng = np.random.default_rng() if rng is None else rng
    p_true = np.atleast_1d(np.asarray(p_true, dtype=float))

    R0 = model.reflectance(p_true, lam_nm, theta_rad)
    out = np.empty((n_trials, p_true.size))

    for i in range(n_trials):
        R_noisy = R0 + rng.normal(0.0, sigma, size=np.shape(R0))
        fr = model.fit(lam_nm, theta_rad, R_noisy, d_grid, dn_grid=dn_grid,
                      sigma=sigma if use_known_sigma else None)
        out[i] = fr.params
    return out
```

판정 기준. 산포가 CRLB와 같으면 추정기가 효율적이고, 크면 정보를 흘리고 있으며, 평균이 참값과 다르면 편향이 있습니다. 실측 결과는 세 자릿수 잡음 범위에서 평균 효율 **1.019**, 편향 < 0.005 nm였습니다.

docstring의 마지막 문단이 이 프로젝트의 정직성 장치입니다. 같은 모델이 데이터를 생성하고 적합하므로(inverse crime), 존재하는 오차원은 주입한 잡음과 옴티마이저뿐입니다. 틀린 분산 모델, 무시된 자연 산화막, 거칠기, 유한 빔 발산 — 모든 모델 형태 오차가 구성상 보이지 않습니다.

따라서 여기서 나온 σ 는 잡음 바닥, 즉 성능의 상한이지 검증된 정확도가 아닙니다. 이 경고를 코드 안에 박아 둔 이유는, 나중에 결과만 인용할 때 스스로를 속이지 않기 위해서입니다. 실험 4가 이 경고를 정량화합니다 — 4 pm 대 1.3 nm, 300배.

13 실험 스크립트가 조작하는 손잡이

네 스크립트는 같은 코어를 쓰되 **서로 다른 변수를 흔듭니다**. 무엇을 흔드는지가 각 실험이 답하는 질문을 정의합니다.

파일	흔드는 손잡이	답하는 질문
exp01	d, λ, θ	반사율이 어떻게 생겼고, 어디에 두께 정보가 있는가
exp02	측정 설계	이 데이터로 d 를 유일하게 정할 수 있는가
exp03	잡음 σ_R	정보 한계에 도달했는가
exp04	생성 모델 $\neq$ 적합 모델	모델이 틀리면 얼마나 틀리는가

exp01~03은 같은 모델로 데이터를 만들고 맞춥니다(inverse crime). **exp04만 이를 깹니다** — 생성 쪽에 계면층이나 거칠기층을 끼우거나, 다른 Si 데이터셋을 쓰거나, 각도를 틀리게 줍니다. 그 세 줄이 4 pm과 1.3 nm를 갈라놓습니다.

13.1 inverse crime을 깨는 코드

```
# exp04: 생성은 층4, 적합은 층3
n_true = build_n_list(["air", "SiO2", n_constant(2.5), "Si"], lamC)
R = coh_tmm("s", n_true, [INF, D_TRUE, t, INF], thC, lamC)["R"]
fr = fit_to(R) # fit_model 은 air/SiO2/Si 층3 고정
bias = fr.params[0] - D_TRUE
```

4줄이 프로젝트의 결론을 만듭니다. 생성 스택에 층 하나를 끼워 넣고 적합 모델은 그대로 두면, 적합기는 계면층이 만든 위상 섭동을 조절 가능한 유일한 파라미터인 d 로 흡수할 수밖에 없습니다. 1 nm 계면층 $\rightarrow$ 0.745 nm 편향, 랜덤 바닥의 168배입니다.

같은 패턴으로 거칠기(Bruggerman EMA 층 추가), 굴절률 오프셋(생성 시 $+\Delta n$), 참조 데이터(생성은 Aspnes, 적합은 Green), 각도 오차(생성 각도에 $+\delta$)를 각각 격리해 측정합니다. **한 번에 하나씩 흔드는 것이** 요점입니다 — 그래야 예산의 각 항목이 무엇에서 왔는지 추적됩니다.

Part III — 전체 원본 소스

14 축약 없는 소스 코드

Part II에서 지면상 접었던 docstring을 포함해 저장소의 모든 소스를 그대로 싣습니다. 줄 번호는 각 파일의 실제 줄 번호입니다.

14.1 omf/core.py — TMM 엔진 (260줄)

```

"""
omf.core -- Isotropic 2x2 transfer-matrix method (TMM) for multilayer thin films.

Sign / phase convention
-----
Time dependence      : exp(-i omega t)
Complex refractive index : n_tilde = n + i k, with k >= 0 for an absorbing medium
Forward-propagating wave : exp(+i k_z z)

With this pair of choices a wave travelling in +z inside an absorbing medium
carries the factor exp(i Re(k_z) z) * exp(-Im(k_z) z), so physical decay requires
Im(k_z) >= 0. That single inequality is the "only" branch rule needed and it is
enforced explicitly in 'kz_from_index' (see '_fix_branch').

Reference
-----
S. J. Byrnes, "Multilayer optical calculations", arXiv:1603.02720.
The layer-matrix construction, the p-polarisation Fresnel convention and the
transmittance prefactors below follow that paper.

Layer indexing
-----
Layer 0      : semi-infinite incident (superstrate) medium
Layer 1..N-2: finite-thickness films
Layer N-1    : semi-infinite exit (substrate) medium

Thicknesses of layers 0 and N-1 must be np.inf.
"""

from __future__ import annotations
import numpy as np

__all__ = [
    "kz_from_index",
    "fresnel_rt",
    "coh_tmm",
    "snell_angles",
]

# Branch-selection tolerance. Values of |Im(k_z)| below this are treated as
# exactly zero (a lossless propagating wave), for which we instead require
# Re(k_z) > 0.
_BRANCH_TOL = 1e-10

# -----
# 1. Transverse wavevector and branch selection
# -----
def _fix_branch(kz: np.ndarray) -> np.ndarray:
    """Force Im(k_z) >= 0; for the marginal lossless case force Re(k_z) > 0.

    Physics: with exp(-i omega t) and exp(+i k_z z), the amplitude of the
    forward wave is exp(-Im(k_z) z). Choosing the root with Im(k_z) < 0 would
    describe a wave that "grows" with propagation distance -- unphysical for a
    passive medium, and numerically catastrophic for thick absorbing layers
    (overflow in exp(+i k_z d)).

    """
    kz = np.asarray(kz, dtype=complex)
    bad = (kz.imag < -_BRANCH_TOL) | (
        (np.abs(kz.imag) <= _BRANCH_TOL) & (kz.real < 0.0)
    )
    return np.where(bad, -kz, kz)

def kz_from_index(n: np.ndarray, lam_vac: np.ndarray, kx: np.ndarray) -> np.ndarray:
    """Normal component of the wavevector.

    Parameters
    -----
    n      : complex refractive index, shape (... , N)
    lam_vac : vacuum wavelength (same length unit as thicknesses), shape (...)
    kx     : conserved in-plane wavevector, shape (...)

    Returns
    -----
    k_z of shape (... , N), branch-corrected.

    Note
    ----
    k_x is conserved across every interface (translational invariance in the
    plane). This "is" Snell's law -- writing it as k_x conservation rather than
    n1 sin(th1) = n2 sin(th2) avoids all complex-angle bookkeeping and handles
    total internal reflection and absorbing media without special cases.

    """
    n = np.asarray(n, dtype=complex)
    k0 = 2.0 * np.pi / np.asarray(lam_vac, dtype=float)

```

```

88     kz2 = (n * k0[..., None]) ** 2 - np.asarray(kx, dtype=complex)[..., None] ** 2
89     return _fix_branch(np.sqrt(kz2))
90
91
92 def snell_angles(n: np.ndarray, lam_vac: np.ndarray, kx: np.ndarray) -> np.ndarray:
93     """Complex propagation angles, provided for reporting only.
94
95     The solver never needs these -- it works directly with k_z.
96     """
97     n = np.asarray(n, dtype=complex)
98     k0 = 2.0 * np.pi / np.asarray(lam_vac, dtype=float)
99     return np.arcsin(np.asarray(kx, dtype=complex)[..., None] / (n * k0[..., None]))
100
101
102 # -----
103 # 2. Interface (Fresnel) coefficients
104 # -----
105 def fresnel_rt(pol: str, n_i, n_j, kz_i, kz_j):
106     """Amplitude reflection / transmission coefficients for interface i -> j.
107
108     Written in terms of k_z instead of cos(theta): n_m cos(theta_m) = k_zm / k0,
109     so the common factor k0 cancels from every ratio below.
110
111     s-polarisation (TE, E parallel to the interface):
112         r = (kz_i - kz_j) / (kz_i + kz_j)
113         t = 2 * kz_i / (kz_i + kz_j)
114
115     p-polarisation (TM, H parallel to the interface):
116         r = (n_j^2 * kz_i - n_i^2 * kz_j) / (n_j^2 * kz_i + n_i^2 * kz_j)
117         t = 2 * n_i * n_j * kz_i / (n_j^2 * kz_i + n_i^2 * kz_j)
118
119     The p-polarisation forms follow Byrnes' convention, in which r_p > 0 below
120     the Brewster angle. The *sign* convention for r_p is a genuine convention
121     (it depends on how the positive direction of E_p is defined after
122     reflection); |r_p|^2 and hence R_p is convention independent.
123     """
124     n_i = np.asarray(n_i, dtype=complex)
125     n_j = np.asarray(n_j, dtype=complex)
126
127     if pol == "s":
128         denom = kz_i + kz_j
129         r = (kz_i - kz_j) / denom
130         t = 2.0 * kz_i / denom
131     elif pol == "p":
128         denom = n_j**2 * kz_i + n_i**2 * kz_j
129         r = (n_j**2 * kz_i - n_i**2 * kz_j) / denom
130         t = 2.0 * n_i * n_j * kz_i / denom
131     else:
132         raise ValueError("pol must be 's' or 'p'")
133     return r, t
134
135
136 # -----
137 # 3. Coherent transfer matrix
138 # -----
139 def _matmul(a, b):
140     """Batched 2x2 matrix product over trailing axes."""
141     return np.einsum("...ij,...jk->...ik", a, b)
142
143
144 def coh_tmm(pol, n_list, d_list, th_0, lam_vac, want_fields=False):
145     """Fully coherent TMM for a stack of isotropic layers.
146
147     Parameters
148     -----
149     pol      : 's' or 'p'
150     n_list    : complex index array, shape (...) -- broadcasts against the
151                 batch index implied by 'th_0' and 'lam_vac'
152     d_list    : thickness array, shape (N,); d_list[0] and d_list[-1] must be inf
153     th_0      : incidence angle in the (real, lossless) medium 0, radians, shape (...)
154     lam_vac   : vacuum wavelength, same length unit as d_list, shape (...)
155
156     Returns
157     -----
158     dict with keys r, t, R, T, A (A = 1 - R - T, the total absorbed fraction)
159
160     Vectorisation
161     -----
162     Everything is broadcast: a call with lam_vac of shape (M,) and th_0 of shape
163     (1,) evaluates M wavelengths in a single pass. The only Python-level loop
164     runs over layers (N is small, typically 3-5), never over data points.
165     """
166     d_list = np.asarray(d_list, dtype=float)
167     n_list = np.asarray(n_list, dtype=complex)
168     N = d_list.size
169
170     if n_list.shape[-1] != N:
171         raise ValueError(f"n_list last axis ({n_list.shape[-1]}) != len(d_list) ({N})")
172     if not (np.isinf(d_list[0]) and np.isinf(d_list[-1])):
173         raise ValueError("d_list[0] and d_list[-1] must be np.inf (semi-infinite media)")

```

```

170 if N >= 3 and not np.all(np.isfinite(d_list[1:-1])):
171     raise ValueError("interior thicknesses must be finite")
180
181 th_0 = np.asarray(th_0, dtype=float)
182 lam_vac = np.asarray(lam_vac, dtype=float)
183
184 # ---- broadcast batch shape -----
185 batch = np.broadcast_shapes(th_0.shape, lam_vac.shape, n_list.shape[:-1])
186 th_0 = np.broadcast_to(th_0, batch)
187 lam_vac = np.broadcast_to(lam_vac, batch)
188 n_list = np.broadcast_to(n_list, batch + (N,))
189
190 k0 = 2.0 * np.pi / lam_vac
191 n0 = n_list[..., 0]
192 if np.max(np.abs(n0.imag)) > 1e-12:
193     raise ValueError("the incident medium must be lossless (real n) for R/T to be well defined")
194
195 # In-plane wavevector: conserved through the whole stack.
196 kx = (n0.real * k0) * np.sin(th_0)
197
198 kz = kz_from_index(n_list, lam_vac, kx)          # (... , N)
199
200 # ---- propagation phases for interior layers -----
201 # delta_j = k_zj d_j. Im(delta) >= 0 guaranteed by the branch rule, so
202 # exp(+i delta) never overflows.
203 delta = kz[..., 1:-1] * d_list[1:-1]            # (... , N-2)
204
205 # ---- interface coefficients -----
206 r_if, t_if = fresnel_rt(pol, n_list[..., :-1], n_list[..., 1:],
207                          kz[..., :-1], kz[..., 1:]) # (... , N-1)
208
209 # ---- assemble M = D_01 * (P_1 D_12) * (P_2 D_23) * ... -----
210 def D(idx):
211     """Interface matrix (1/t)[[1, r],[r, 1]] for interface `idx`."""
212     r = r_if[..., idx]
213     t = t_if[..., idx]
214     m = np.empty(batch + (2, 2), dtype=complex)
215     m[..., 0, 0] = 1.0
216     m[..., 0, 1] = r
217     m[..., 1, 0] = r
218     m[..., 1, 1] = 1.0
219     return m / t[..., None, None]
220
221 def P(j):
222     """Propagation matrix diag(exp(-i delta), exp(+i delta)) for film `j`."""
223     dl = delta[..., j]
224     m = np.zeros(batch + (2, 2), dtype=complex)
225     m[..., 0, 0] = np.exp(-1j * dl)
226     m[..., 1, 1] = np.exp(+1j * dl)
227     return m
228
229 M = D(0)
230 for j in range(N - 2):          # one propagation + one interface each
231     M = _matmul(M, P(j))
232     M = _matmul(M, D(j + 1))
233
234 # [E0+, E0-]^T = M [Ef+, 0]^T  ->  t = 1/M11,  r = M21/M11
235 r = M[..., 1, 0] / M[..., 0, 0]
236 t = 1.0 / M[..., 0, 0]
237
238 R = np.abs(r) ** 2
239
240 # ---- transmittance: needs the ratio of longitudinal Poynting flux -----
241 # s-pol : T = |t|^2 Re(n_f cos th_f) / Re(n_0 cos th_0)
242 # p-pol : T = |t|^2 Re(n_f cos th_f^*) / Re(n_0 cos th_0^*)
243 # The conjugation in the p case is NOT cosmetic: for an absorbing exit medium
244 # the two prefactors differ, and using the s-form for p violates energy
245 # conservation. Rewritten with n cos th = k_z / k0:
246 # Re(n cos th) = Re(k_z)/k0
247 # Re(n cos th^*) = Re(n^2 k_z^*) / (k0 |n|^2)
248 nf, kzf = n_list[..., -1], kz[..., -1]
249 if pol == "s":
250     num, den = kzf.real, kz[..., 0].real
251 else:
252     num = (nf**2 * np.conj(kzf)).real / np.abs(nf) ** 2
253     den = (n0**2 * np.conj(kz[..., 0])).real / np.abs(n0) ** 2
254 T = np.abs(t) ** 2 * (num / den)
255
256 out = {"r": r, "t": t, "R": R, "T": T, "A": 1.0 - R - T, "kz": kz, "kx": kx}
257 if want_fields:
258     out["M"] = M
259 return out

```

14.2 omf/materials.py — 분산 모델 (191줄)

```

"""
omf.materials -- dispersion models.

```

```

1 All functions take wavelength in **nanometres** and return the complex index
2 n_tilde = n + i k consistent with the exp(-i omega t) convention of omf.core.
3
4 Two classes of model are used, for a reason worth stating explicitly:
5
6 * **Sellmeier** (SiO2, BK7): a physically-motivated closed form derived from a
7   sum of Lorentz oscillators in the limit of negligible damping,
8    $n^2 - 1 = \sum_j B_j \lambda^2 / (\lambda^2 - C_j)$ ,
9   valid only in the transparency window between the UV and IR absorption
10  bands. It gives k = 0 by construction, which is an excellent approximation
11  for fused silica in the visible ( $\alpha < 1e-4$  /cm).
12
13 * **Tabulated n,k** (Si): silicon has strong interband absorption (E1 at 3.4 eV,
14   E2 at 4.25 eV) in the visible/UV, so no low-order analytic form works. We
15   interpolate a Kramers-Kronig-consistent measured data set.
16   """
17
18 from __future__ import annotations
19
20 import os
21 import numpy as np
22
23 __all__ = [
24     "n_air", "n_vacuum", "n_sio2", "n_bk7", "n_si", "n_si_aspnes", "n_constant",
25     "MATERIALS",
26 ]
27
28 _DATA_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "data")
29
30 # -----
31 # Sellmeier models
32 # -----
33 def _sellmeier(lam_nm, B, C):
34     """ $n^2 = 1 + \sum_j B_j \lambda^2 / (\lambda^2 - C_j)$ , with lam in micrometres."""
35     lam_um2 = (np.asarray(lam_nm, dtype=float) / 1000.0) ** 2
36     n2 = 1.0
37     for b, c in zip(B, C):
38         n2 = n2 + b * lam_um2 / (lam_um2 - c)
39     return np.sqrt(n2).astype(complex)
40
41 def n_sio2(lam_nm):
42     """Fused silica, 20 C. Malitson, J. Opt. Soc. Am. 55, 1205 (1965).
43
44     Valid 210 nm - 3.71 um. k is taken as exactly zero.
45
46     Caveat for metrology: "thermal" SiO2 grown on Si is not identical to bulk
47     fused silica. Densification and stress typically raise n by ~0.002-0.006 in
48     the visible. Treating n_film as a free fit parameter (see fitter.py) is one
49     way to absorb this; assuming Malitson exactly is a systematic error source.
50     """
51     B = (0.6961663, 0.4079426, 0.8974794)
52     C = (0.0684043**2, 0.1162414**2, 9.896161**2)
53     return _sellmeier(lam_nm, B, C)
54
55 def n_bk7(lam_nm):
56     """N-BK7 borosilicate crown (SCHOTT). Valid 300 nm - 2.5 um.
57
58     Used here as a stand-in for a microscope coverslip. Real coverslips are
59     usually D263 or B270 (n_d ~ 1.5230 vs 1.5168 for N-BK7), so this is a ~0.4%
60     index approximation -- acceptable for a Brewster-angle demonstration, not
61     for absolute thickness metrology.
62     """
63     B = (1.03961212, 0.231792344, 1.01046945)
64     C = (0.00600069867, 0.0200179144, 103.560653)
65     return _sellmeier(lam_nm, B, C)
66
67 def n_air(lam_nm):
68     """Air at STP, treated as non-dispersive.
69
70     n_air = 1.000293 at 589 nm; the dispersion across the visible is ~3e-6,
71     far below any effect we can resolve, so a constant is used.
72     """
73     return np.full(np.shape(lam_nm), 1.000293, dtype=complex)
74
75 def n_vacuum(lam_nm):
76     return np.full(np.shape(lam_nm), 1.0, dtype=complex)
77
78 def n_constant(value):
79     """Factory for a dispersionless material of given complex index."""
80     def f(lam_nm):
81         return np.full(np.shape(lam_nm), complex(value), dtype=complex)
82     return f
83
84 MATERIALS = {
85     "SiO2": n_sio2,
86     "BK7": n_bk7,
87     "Air": n_air,
88     "Vacuum": n_vacuum,
89     "Constant": n_constant,
90 }
91
92 f = n_constant(0)
93

```

```

98 # -----
99 # Tabulated silicon
100 # -----
101 def _load_si_table(fname="Si_Green2008.csv"):
102     path = os.path.join(_DATA_DIR, fname)
103     rows = []
104     with open(path) as fh:
105         for line in fh:
106             line = line.strip()
107             if not line or line.startswith("#") or line[0].isalpha():
108                 continue
109             # skip provenance comments and the column header
110             rows.append([float(x) for x in line.split(",")])
111     arr = np.asarray(rows, dtype=float)
112     lam, n, k = arr[:, 0], arr[:, 1], arr[:, 2]
113     order = np.argsort(lam)
114     return lam[order], n[order], k[order]
115
116 _SI_LAM, _SI_N, _SI_K = _load_si_table()
117 SI_RANGE_NM = (float(_SI_LAM[0]), float(_SI_LAM[-1]))
118
119 def n_si(lam_nm):
120     """Intrinsic crystalline silicon at 300 K.
121
122     Data: M. A. Green, Sol. Energ. Mat. Sol. Cells 92, 1305 (2008), a
123     Kramers-Kronig-consistent compilation. Retrieved from the
124     refractiveindex.info database (CCO 1.0). Range 250-1100 nm.
125
126     Interpolation: n linearly, k in log space. k spans five decades over this
127     range (3.7 at 250 nm down to 3e-5 at 1100 nm) because silicon's gap is
128     indirect -- near the band edge absorption requires phonon assistance and
129     falls off smoothly by orders of magnitude. Linear interpolation of such a
130     quantity between sparse grid points is badly wrong; log interpolation is the
131     correct choice for an exponential-like tail.
132     """
133     lam = np.asarray(lam_nm, dtype=float)
134     if np.any(lam < _SI_LAM[0] - 1e-9) or np.any(lam > _SI_LAM[-1] + 1e-9):
135         raise ValueError(
136             f"n_si: wavelength outside tabulated range {_SI_RANGE_NM} nm"
137         )
138     n = np.interp(lam, _SI_LAM, _SI_N)
139     floor = 1e-12
140     logk = np.interp(lam, _SI_LAM, np.log(np.maximum(_SI_K, floor)))
141     k = np.exp(logk)
142     return (n + 1j * k).astype(complex)
143
144 _ASP_LAM, _ASP_N, _ASP_K = _load_si_table("Si_Aspnes1983.csv")
145
146 def n_si_aspnes(lam_nm):
147     """Crystalline silicon, D. E. Aspnes & A. A. Studna, PRB 27, 985 (1983).
148
149     An *independent* determination (spectroscopic ellipsometry on chemically
150     stripped surfaces), valid 206-826 nm. Provided deliberately as a second
151     opinion: the difference between this and Green-2008 is not numerical noise
152     but a genuine reference-data disagreement, and propagating that difference
153     through the fit is the honest way to quote a systematic uncertainty on
154     thickness. See scripts/exp04_systematics.py.
155     """
156     lam = np.asarray(lam_nm, dtype=float)
157     if np.any(lam < _ASP_LAM[0] - 1e-9) or np.any(lam > _ASP_LAM[-1] + 1e-9):
158         raise ValueError(
159             f"n_si_aspnes: outside tabulated range "
160             f"({_ASP_LAM[0]}, {_ASP_LAM[-1]}) nm"
161         )
162     n = np.interp(lam, _ASP_LAM, _ASP_N)
163     k = np.exp(np.interp(lam, _ASP_LAM, np.log(np.maximum(_ASP_K, 1e-12))))
164     return (n + 1j * k).astype(complex)
165
166 MATERIALS = {
167     "air": n_air,
168     "vacuum": n_vacuum,
169     "SiO2": n_sio2,
170     "BK7": n_bk7,
171     "Si": n_si,
172     "Si_Aspnes": n_si_aspnes,
173 }
174
175 def build_n_list(names_or_values, lam_nm):
176     """Assemble an (...) index array from a list of material names/values.
177
178     Each entry may be a string key of MATERIALS, a callable f(lam_nm), or a
179     complex constant.
180     """
181     lam = np.asarray(lam_nm, dtype=float)
182     cols = []
183     for item in names_or_values:

```

```

184         if isinstance(item, str):
185             cols.append(np.broadcast_to(MATERIALS[item](lam), lam.shape))
186         elif callable(item):
187             cols.append(np.broadcast_to(item(lam), lam.shape))
188         else:
189             cols.append(np.full(lam.shape, complex(item), dtype=complex))
190     return np.stack(cols, axis=-1)
191

```

14.3 omf/fitter.py — 역문제 솔버 (222줄)

```

"""
omf.fitter -- the inverse problem: recover film thickness (and optionally index)
from measured reflectance.

Why this is not a trivial least-squares problem
-----
For a transparent film on a substrate the reflectance is, to leading order,

    R(d) ~ A + B cos(2 delta),    delta = (2 pi / lam) n_f d cos(theta_f)

i.e. **periodic in d**. A film that is thicker by one full order,

    Delta d = lam / (2 n_f cos theta_f)  (~139 nm for SiO2 at 405 nm, normal incidence)

produces an almost identical single-wavelength reflectance. The cost function
therefore has a comb of near-degenerate local minima, and a gradient-based
optimiser started at an arbitrary guess will simply fall into whichever tooth of
the comb it happens to be nearest. This is the classic **fringe-order
ambiguity** of thin-film metrology.

Two things break the degeneracy, and the code exposes both:

1. **Coarse grid pre-scan.** Evaluate the cost on a dense grid spanning the
   plausible thickness range, take the global minimum as the starting point,
   then refine locally. Cheap because the forward model is vectorised.
2. **Spectroscopic / angle-resolved data.** The fringe spacing itself is
   wavelength-dependent (delta ∝ 1/lam), so a broadband measurement pins the
   absolute order. A single-wavelength, single-angle measurement genuinely
   cannot -- no algorithm fixes that; it is an information-content limit, not an
   optimiser limit.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Callable, Sequence

import numpy as np
from scipy.optimize import least_squares

from .core import coh_tmm
from .materials import build_n_list

__all__ = ["FilmModel", "FitResult"]

@dataclass
class FitResult:
    """Outcome of a fit, with uncertainty propagated from the Jacobian."""
    params: np.ndarray          # best-fit parameter vector
    names: list                 # parameter names
    sigma: np.ndarray           # 1-sigma from the covariance diagonal
    cov: np.ndarray             # parameter covariance matrix
    rms: float                  # RMS of the residual
    coarse_best: np.ndarray     # starting point from the grid pre-scan
    success: bool = True
    nfev: int = 0
    extra: dict = field(default_factory=dict)

    def __repr__(self):
        lines = [f"FitResult(rms={self.rms:.3e}, nfev={self.nfev})"]
        for nm, p, s in zip(self.names, self.params, self.sigma):
            lines.append(f"    {nm:>10s} = {p:12.5f} +/- {s:.5f}")
        return "\n".join(lines)

    @property
    def correlation(self):
        d = np.sqrt(np.diag(self.cov))
        outer = np.outer(d, d)
        with np.errstate(invalid="ignore", divide="ignore"):
            return np.where(outer > 0, self.cov / outer, np.nan)

class FilmModel:
    """A stack with one free-thickness film and an optional index offset.

    Parameters
    -----

```

```

77 materials : sequence, length N
78     Entries are material names ('air', 'SiO2', 'Si'), callables f(lam_nm),
79     or complex constants. Index 0 = superstrate, index -1 = substrate.
80 d_nm : sequence, length N
81     Thicknesses in nm; d_nm[0] and d_nm[-1] must be np.inf.
82 fit_layer : int
83     Index of the layer whose thickness is a free parameter.
84 fit_dn : bool
85     If True, add a constant real offset 'dn' to the fit layer's index.
86     This is the standard remedy for the difference between bulk fused silica
87     and thermal oxide -- but see the correlation analysis: 'd' and 'dn' are
88     strongly anti-correlated for single-angle data, because the optical path
89     n*d is what the fringes actually measure.
90 pol : {'s', 'p', 'unpol'}
91 """
92
93 def __init__(self, materials: Sequence, d_nm: Sequence[float],
94              fit_layer: int = 1, fit_dn: bool = False, pol: str = "s"):
95     self.materials = list(materials)
96     self.d_nm = np.asarray(d_nm, dtype=float)
97     self.fit_layer = fit_layer
98     self.fit_dn = fit_dn
99     self.pol = pol
100
101     if pol not in ("s", "p", "unpol"):
102         raise ValueError("pol must be 's', 'p' or 'unpol'")
103     if not (np.isinf(self.d_nm[0]) and np.isinf(self.d_nm[-1])):
104         raise ValueError("outer layers must have infinite thickness")
105     if not np.isfinite(self.d_nm[fit_layer]):
106         raise ValueError("fit_layer must be a finite-thickness film")
107
108     self.names = ["d_nm"] + (["dn"] if fit_dn else [])
109
110 # -----
111 # forward model
112 # -----
113 def reflectance(self, params, lam_nm, theta_rad):
114     """R(lam, theta) for the given parameter vector.
115
116     'lam_nm' and 'theta_rad' are broadcast against each other, so an
117     (M,) shaped lam with scalar theta costs one vectorised pass.
118     """
119     params = np.atleast_1d(np.asarray(params, dtype=float))
120     d = self.d_nm.copy()
121     d[self.fit_layer] = params[0]
122
123     lam = np.asarray(lam_nm, dtype=float)
124     n_list = build_n_list(self.materials, lam)
125
126     if self.fit_dn:
127         n_list = n_list.copy()
128         n_list[... , self.fit_layer] += params[1]
129
130     if self.pol == "unpol":
131         rs = coh_tmm("s", n_list, d, theta_rad, lam)["R"]
132         rp = coh_tmm("p", n_list, d, theta_rad, lam)["R"]
133         return 0.5 * (rs + rp)
134     return coh_tmm(self.pol, n_list, d, theta_rad, lam)["R"]
135
136 # -----
137 # coarse grid pre-scan
138 # -----
139 def coarse_scan(self, lam_nm, theta_rad, R_meas, d_grid, dn_grid=None):
140     """Evaluate SSR on a grid and return the global minimiser.
141
142     Complexity: len(d_grid) forward evaluations, each already vectorised
143     over all data points. For a 2000-point thickness grid and 400
144     wavelengths this is ~8e5 TMM evaluations, well under a second.
145     """
146     d_grid = np.asarray(d_grid, dtype=float)
147     dn_grid = np.array([0.0]) if dn_grid is None else np.asarray(dn_grid, float)
148     R_meas = np.asarray(R_meas, dtype=float)
149
150     ssr = np.empty((d_grid.size, dn_grid.size))
151     for i, d in enumerate(d_grid):
152         for j, dn in enumerate(dn_grid):
153             p = (d, dn) if self.fit_dn else (d,)
154             resid = self.reflectance(p, lam_nm, theta_rad) - R_meas
155             ssr[i, j] = float(np.sum(resid**2))
156
157     i, j = np.unravel_index(np.argmin(ssr), ssr.shape)
158     best = [d_grid[i]] + ([dn_grid[j]] if self.fit_dn else [])
159     return np.array(best), ssr, d_grid, dn_grid
160
161 # -----
162 # full fit
163 # -----
164 def fit(self, lam_nm, theta_rad, R_meas, d_grid,
165        dn_grid=None, sigma=None, bounds=None, verbose=False):
166     """Coarse pre-scan followed by Levenberg-Marquardt-style refinement.

```

```

168
169 Parameters
170 -----
171 sigma : float or array or None
172     Known 1-sigma measurement noise on R. If given, the covariance is
173     J^T J scaled by sigma^2 (an "a priori" estimate). If None, the
174     covariance is scaled by the reduced chi-squared of the fit
175     (an "a posteriori" estimate), which is what you must use when the
176     noise level is not independently known.
177
178 """
179 R_meas = np.asarray(R_meas, dtype=float)
180 w = 1.0 if sigma is None else 1.0 / np.asarray(sigma, dtype=float)
181
182 p0, ssr_grid, dg, dng = self.coarse_scan(
183     lam_nm, theta_rad, R_meas, d_grid, dn_grid)
184
185 def resid(p):
186     return (self.reflectance(p, lam_nm, theta_rad) - R_meas) * w
187
188 if bounds is None:
189     lo = [float(np.min(d_grid))] + ([-0.5] if self.fit_dn else [])
190     hi = [float(np.max(d_grid))] + ([+0.5] if self.fit_dn else [])
191     bounds = (lo, hi)
192
193 res = least_squares(resid, p0, bounds=bounds, method="trf",
194                     xtol=1e-14, ftol=1e-14, gtol=1e-14,
195                     verbose=2 if verbose else 0)
196
197 m, npar = res.fun.size, res.x.size
198 J = res.jac
199 JTJ = J.T @ J
200 try:
201     cov = np.linalg.inv(JTJ)
202 except np.linalg.LinAlgError:
203     cov = np.full((npar, npar), np.nan)
204
205 if sigma is None:
206     dof = max(m - npar, 1)
207     chi2_red = float(np.sum(res.fun**2)) / dof
208     cov = cov * chi2_red
209 # if sigma was supplied the residuals are already whitened, so
210 # inv(J^T J) is directly the covariance.
211
212 rms = float(np.sqrt(np.mean(
213     (self.reflectance(res.x, lam_nm, theta_rad) - R_meas) ** 2)))
214
215 return FitResult(
216     params=res.x, names=self.names,
217     sigma=np.sqrt(np.abs(np.diag(cov))),
218     cov=cov, rms=rms, coarse_best=p0,
219     success=bool(res.success), nfev=int(res.nfev),
220     extra={"ssr_grid": ssr_grid, "d_grid": dg, "dn_grid": dng},
221 )

```

14.4 omf/uncertainty.py — 불확도 분석 (105줄)

```

1 """
2 omf.uncertainty -- how well "can" a thickness be determined, and how well "is" it?
3
4 The distinction matters, and it is the part of a metrology project that a
5 fitting routine alone does not answer.
6
7 Cramer-Rao Lower bound (CRLB)
8 -----
9 For measurements R_i with independent Gaussian noise of standard deviation
10 sigma_i, and a model R_i(p) depending on parameters p, the Fisher information
11 matrix is
12
13 F_ab = sum_i (1/sigma_i^2) (dR_i/dp_a)(dR_i/dp_b)
14
15 and "any" unbiased estimator obeys Cov(p) >= F^{-1} (matrix inequality). So
16
17 sigma_d >= sqrt( (F^{-1})_dd ).
18
19 This is a statement about the "experiment", not the algorithm: it says how much
20 information about d is physically present in the data. If the measured spread
21 sits at the CRLB, the estimator is efficient and the only way to do better is to
22 change the measurement (more points, lower noise, wider spectral range, a
23 different angle). If the measured spread is far above it, the estimator is
24 wasteful.
25
26 Note the crucial asymmetry: the CRLB bounds the "variance", i.e. the random
27 error. It says nothing at all about "bias" from a wrong model. A fit can sit
28 exactly at the CRLB and still be 3 nm wrong because the true sample has a native
29 oxide the model omits. Random precision and systematic accuracy are separate
30 budgets (see scripts/exp04).
31 """
32

```

```

34 from __future__ import annotations
35
36 import numpy as np
37
38 __all__ = ["numerical_jacobian", "fisher_information", "crlb", "monte_carlo_fit"]
39
40 def numerical_jacobian(f, p, steps):
41     """Central-difference Jacobian dF_i/dp_a of a vector-valued model.
42
43     `steps` sets the finite-difference step per parameter. Central differences
44     give  $O(h^2)$  truncation error; with  $h$  chosen  $\sim 1e-4$  of the parameter scale the
45     total error is comfortably below the noise levels considered here.
46     """
47     p = np.asarray(p, dtype=float)
48     steps = np.atleast_1d(np.asarray(steps, dtype=float))
49     cols = []
50     for a in range(p.size):
51         h = steps[a]
52         pp, pm = p.copy(), p.copy()
53         pp[a] += h
54         pm[a] -= h
55         cols.append((np.asarray(f(pp)) - np.asarray(f(pm))) / (2.0 * h))
56     return np.stack(cols, axis=-1) # (M, npar)
57
58 def fisher_information(J, sigma):
59     """F = J^T W J with W = diag(1/sigma^2)."""
60     sigma = np.broadcast_to(np.asarray(sigma, dtype=float), (J.shape[0],))
61     Jw = J / sigma[:, None]
62     return Jw.T @ Jw
63
64 def crlb(J, sigma):
65     """Return (sigma_lower_bound_vector, inverse Fisher matrix)."""
66     F = fisher_information(J, sigma)
67     Finv = np.linalg.inv(F)
68     return np.sqrt(np.abs(np.diag(Finv))), Finv
69
70 def monte_carlo_fit(model, lam_nm, theta_rad, p_true, sigma, n_trials,
71                     d_grid, dn_grid=None, rng=None, use_known_sigma=True):
72     """Synthesise noisy data 'n_trials' times and refit; return the estimates.
73
74     This is the empirical counterpart of the CRLB. Comparing the two is the
75     single most informative validation a fitting pipeline can carry:
76
77     * spread == CRLB -> estimator efficient, measurement-limited
78     * spread > CRLB -> optimiser or parameterisation is losing information
79     * mean != truth -> bias; with the same model on both sides this can only
80       come from the estimator (nonlinearity, bounds), since
81       the model is exact by construction
82
83     IMPORTANT (inverse crime): because 'model' generates the data *and* fits it,
84     the only error sources present are the injected noise and the optimiser.
85     Every model-form error -- wrong dispersion, ignored native oxide, roughness,
86     finite beam divergence -- is invisible here. The resulting sigma is a
87     "noise floor", i.e. an upper bound on achievable performance, never a
88     validated accuracy.
89     """
90     rng = np.random.default_rng() if rng is None else rng
91     p_true = np.atleast_1d(np.asarray(p_true, dtype=float))
92
93     R0 = model.reflectance(p_true, lam_nm, theta_rad)
94     out = np.empty((n_trials, p_true.size))
95
96     for i in range(n_trials):
97         R_noisy = R0 + rng.normal(0.0, sigma, size=np.shape(R0))
98         fr = model.fit(lam_nm, theta_rad, R_noisy, d_grid, dn_grid=dn_grid,
99                       sigma=sigma if use_known_sigma else None)
100         out[i] = fr.params
101     return out

```

14.5 omf/_init_.py — 패키지 인터페이스 (22줄)

```

1 """optical-metrology-fitter (omf)
2
3 A transfer-matrix forward model and inverse solver for thin-film optical
4 metrology, with an explicit uncertainty budget.
5
6 Convention:  $\exp(-i \omega t)$ ,  $n_{\text{tilde}} = n + i k$ ,  $\text{Im}(k_z) \geq 0$ .
7 """
8
9 from .core import coh_tmm, fresnel_rt, kz_from_index, snell_angles
10 from .materials import (n_air, n_vacuum, n_sio2, n_bk7, n_si, n_si_aspn,
11                        n_constant, build_n_list, MATERIALS)
12 from .fitter import FilmModel, FitResult
13 from .uncertainty import numerical_jacobian, fisher_information, crlb, monte_carlo_fit

```

```

14 __version__ = "0.3.0"
15 __all__ = [
16     "coh_tmm", "fresnel_rt", "kz_from_index", "snell_angles",
17     "n_air", "n_vacuum", "n_sio2", "n_bk7", "n_si", "n_si_aspnes", "n_constant",
18     "build_n_list", "MATERIALS",
19     "FilmModel", "FitResult",
20     "numerical_jacobian", "fisher_information", "crlb", "monte_carlo_fit",
21 ]

```

14.6 tests/test_core.py — 검증 스위트 (34 항목) (416줄)

```

1 """
2 Validation suite for omf.core.
3
4 Three tiers, in increasing order of evidential weight:
5
6 A. Internal consistency -- energy conservation, branch selection, broadcasting.
7   Catches coding errors but cannot catch a wrong "convention".
8 B. Analytic limits -- Fresnel, Airy, Brewster, absentee layer, quarter-wave AR,
9   total internal reflection. These have closed forms derived independently of
10  the matrix machinery, so agreement tests the physics, not just the code.
11 C. External cross-check -- against `tmm` (S. J. Byrnes), the reference
12  implementation of the same paper this code follows. This is the strongest
13  single test: an independent author, independent code path, same physics.
14
15 Run:  python tests/test_core.py      (or: pytest tests/test_core.py)
16 """
17
18 from __future__ import annotations
19
20 import os
21 import sys
22
23 import numpy as np
24
25 sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), ".."))
26
27 from omf.core import coh_tmm, fresnel_rt, kz_from_index      # noqa: E402
28 from omf.materials import n_si, n_sio2, build_n_list        # noqa: E402
29
30 INF = np.inf
31 _RESULTS = []
32
33 def check(name, condition, detail=""):
34     _RESULTS.append((name, bool(condition), detail))
35     flag = "PASS" if condition else "FAIL"
36     print(f" [{flag}] {name}" + (f" {detail}" if detail else ""))
37     assert condition, f"{name}: {detail}"
38
39 # =====
40 # TIER A -- internal consistency
41 # =====
42 def test_energy_conservation_lossless():
43     """R + T = 1 for a lossless stack, both polarisations, all angles."""
44     lam = np.linspace(400.0, 800.0, 41)
45     th = np.linspace(0.0, np.deg2rad(85.0), 18)
46     LAM, TH = np.meshgrid(lam, th, indexing="ij")
47     n = build_n_list(["air", "SiO2", 1.60], LAM)
48     for pol in ("s", "p"):
49         out = coh_tmm(pol, n, [INF, 217.0, INF], TH, LAM)
50         err = np.max(np.abs(out["R"] + out["T"] - 1.0))
51         check(f"A1 energy conservation, lossless, {pol}-pol",
52               err < 1e-12, f"max|R+T-1| = {err:.2e}")
53
54 def test_energy_conservation_absorbing_film():
55     """R + T + A = 1 with an absorbing film and a lossless substrate.
56
57     This is where a wrong p-polarisation transmittance prefactor shows up: using
58     the s-form Re(n cos th) instead of Re(n cos th*) leaves a residual that grows
59     with angle and with Im(n).
60     """
61     lam = np.linspace(450.0, 750.0, 31)
62     th = np.linspace(0.0, np.deg2rad(80.0), 15)
63     LAM, TH = np.meshgrid(lam, th, indexing="ij")
64     n = build_n_list(["air", 2.10 + 0.35j, 1.52], LAM)
65     for pol in ("s", "p"):
66         out = coh_tmm(pol, n, [INF, 95.0, INF], TH, LAM)
67         A = out["A"]
68         check(f"A2 absorbed fraction non-negative, {pol}-pol",
69               np.min(A) > -1e-12, f"min A = {np.min(A):.2e}")
70         check(f"A3 R+T+A = 1 by construction, {pol}-pol",
71               np.max(np.abs(out["R"] + out["T"] + A - 1.0)) < 1e-12)
72
73 def test_branch_selection():

```

```

75 """Im(k_z) >= 0 in every layer, including evanescent and absorbing ones."""
76 lam = np.array([405.0])
77 th = np.deg2rad(np.linspace(0.0, 89.0, 90))
78 n = build_n_list([1.52, 1.00, "Si"], np.broadcast_to(lam, th.shape))
79 kx = 1.52 * (2 * np.pi / 405.0) * np.sin(th)
80 kz = kz_from_index(n, np.broadcast_to(lam, th.shape), kx)
81 check("A4 Im(k_z) >= 0 everywhere (incl. frustrated TIR)",
82       np.min(kz.imag) > -1e-10, f"min Im(kz) = {np.min(kz.imag):.2e}")
83
84
85
86
87
88
89 def test_broadcasting_matches_loop():
90     """A vectorised call reproduces an element-by-element loop exactly."""
91     lam = np.linspace(400.0, 700.0, 25)
92     th = np.deg2rad(30.0)
93     n = build_n_list(["air", "SiO2", "Si"], lam)
94     vec = coh_tmm("p", n, [INF, 180.0, INF], th, lam)["R"]
95     loop = np.array([
96         coh_tmm("p", build_n_list(["air", "SiO2", "Si"], np.array(L)),
97         [INF, 180.0, INF], th, np.array(L))["R"]
98         for L in lam])
99     check("A5 vectorised == looped", np.max(np.abs(vec - loop)) < 1e-14,
100          f"max diff = {np.max(np.abs(vec - loop)):.2e}")
101
102
103 def test_no_interface():
104     """Identical index throughout -> R = 0, T = 1 (no spurious interfaces)."""
105     lam = np.array([550.0])
106     n = build_n_list([1.5, 1.5, 1.5], lam)
107     for pol in ("s", "p"):
108         out = coh_tmm(pol, n, [INF, 333.0, INF], np.deg2rad(41.0), lam)
109         check(f"A6 index-matched stack is transparent, {pol}-pol",
110              abs(out["R"][0]) < 1e-14 and abs(out["T"][0] - 1.0) < 1e-14)
111
112
113 # =====
114 # TIER B -- analytic limits
115 # =====
116
117 def test_single_interface_fresnel():
118     """Two-layer stack must reproduce the textbook Fresnel coefficients."""
119     n1, n2 = 1.0, 1.52
120     th1 = np.deg2rad(np.linspace(0.0, 89.0, 90))
121     th2 = np.arcsin(n1 * np.sin(th1) / n2)
122     lam = np.full(th1.shape, 589.3)
123     n = build_n_list([n1, n2], lam)
124
125     rs_a = (n1 * np.cos(th1) - n2 * np.cos(th2)) / (n1 * np.cos(th1) + n2 * np.cos(th2))
126     rp_a = (n2 * np.cos(th1) - n1 * np.cos(th2)) / (n2 * np.cos(th1) + n1 * np.cos(th2))
127
128     rs = coh_tmm("s", n, [INF, INF], th1, lam)["r"]
129     rp = coh_tmm("p", n, [INF, INF], th1, lam)["r"]
130     check("B1 s-pol reproduces Fresnel r_s",
131          np.max(np.abs(rs - rs_a)) < 1e-13, f"max diff = {np.max(np.abs(rs - rs_a)):.2e}")
132     check("B2 p-pol reproduces Fresnel r_p",
133          np.max(np.abs(rp - rp_a)) < 1e-13, f"max diff = {np.max(np.abs(rp - rp_a)):.2e}")
134
135
136
137
138
139 def test_brewster_angle():
140     """R_p vanishes at theta_B = arctan(n2/n1)."""
141     n1, n2 = 1.0, 1.52
142     thB = np.arctan(n2 / n1)
143     lam = np.array([589.3])
144     n = build_n_list([n1, n2], lam)
145     Rp = coh_tmm("p", n, [INF, INF], np.array([thB]), lam)["R"][0]
146     Rs = coh_tmm("s", n, [INF, INF], np.array([thB]), lam)["R"][0]
147     check("B3 R_p = 0 at Brewster angle",
148          Rp < 1e-25, f"theta_B = {np.rad2deg(thB):.3f} deg, R_p = {Rp:.2e}")
149     check("B4 R_s != 0 at Brewster angle", Rs > 0.1, f"R_s = {Rs:.4f}")
150
151
152
153
154
155 def test_airy_single_film():
156     """Three-layer stack must reproduce the Airy summation exactly.
157
158     r = (r01 + r12 e^{2i delta}) / (1 + r01 r12 e^{2i delta})
159
160     Tested with a *complex* film index so the geometric-series bookkeeping is
161     exercised, not just the real case.
162     """
163     lam = np.linspace(400.0, 900.0, 51)
164     th0 = np.deg2rad(37.0)
165     nlist = build_n_list(["air", 2.05 + 0.02j, "Si"], lam)
166     d = 240.0
167
168     k0 = 2 * np.pi / lam
169     kx = 1.000293 * k0 * np.sin(th0)
170     kz = kz_from_index(nlist, lam, kx)
171     delta = kz[..., 1] * d
172
173     for pol in ("s", "p"):
174         r01, _ = fresnel_rt(pol, nlist[..., 0], nlist[..., 1], kz[..., 0], kz[..., 1])
175         r12, _ = fresnel_rt(pol, nlist[..., 1], nlist[..., 2], kz[..., 1], kz[..., 2])

```

```

168     ph = np.exp(2j * delta)
169     r_airy = (r01 + r12 * ph) / (1.0 + r01 * r12 * ph)
170     r_tmm = coh_tmm(pol, nlist, [INF, d, INF], th0, lam)["r"]
171     err = np.max(np.abs(r_tmm - r_airy))
172     check(f"B5 TMM == Airy summation, {pol}-pol", err < 1e-13,
173           f"max diff = {err:.2e}")
174
175 def test_absentee_layer():
176     """A half-wave film is optically invisible: R equals the bare substrate.
177
178     delta = pi => e^{2i delta} = 1 => the Airy expression collapses to
179     (r01 + r12)/(1 + r01 r12) = r02. A classic exam identity and a sharp test of
180     the phase factor's sign and magnitude.
181     """
182     lam = 632.8
183     nf = float(n_sio2(lam).real)
184     th0 = np.deg2rad(25.0)
185     kx_over_k0 = 1.000293 * np.sin(th0)
186     cos_f = np.sqrt(1.0 - (kx_over_k0 / nf) ** 2)
187     d_half = lam / (2.0 * nf * cos_f)
188
189     L = np.array([lam])
190     n3 = build_n_list(["air", "SiO2", "Si"], L)
191     n2 = build_n_list(["air", "Si"], L)
192     for pol in ("s", "p"):
193         R_film = coh_tmm(pol, n3, [INF, d_half, INF], th0, L)["R"][0]
194         R_bare = coh_tmm(pol, n2, [INF, INF], th0, L)["R"][0]
195         check(f"B6 half-wave layer is absentee, {pol}-pol",
196               abs(R_film - R_bare) < 1e-12,
197               f"d_1/2 = {d_half:.3f} nm, dR = {abs(R_film - R_bare):.2e}")
198
199 def test_zero_thickness():
200     """d -> 0 must return the bare-substrate result."""
201     lam = np.linspace(400.0, 800.0, 21)
202     th0 = np.deg2rad(55.0)
203     n3 = build_n_list(["air", "SiO2", "Si"], lam)
204     n2 = build_n_list(["air", "Si"], lam)
205     for pol in ("s", "p"):
206         R3 = coh_tmm(pol, n3, [INF, 0.0, INF], th0, lam)["R"]
207         R2 = coh_tmm(pol, n2, [INF, INF], th0, lam)["R"]
208         check(f"B7 zero-thickness film == bare substrate, {pol}-pol",
209               np.max(np.abs(R3 - R2)) < 1e-14)
210
211 def test_quarter_wave_ar():
212     """Perfect single-layer AR coating: n_f = sqrt(n0 ns), d = lam/(4 n_f) -> R = 0."""
213     lam = 550.0
214     n0, ns = 1.0, 2.25
215     nf = np.sqrt(n0 * ns)
216     d = lam / (4.0 * nf)
217     L = np.array([lam])
218     n = build_n_list([n0, nf, ns], L)
219     R = coh_tmm("s", n, [INF, d, INF], 0.0, L)["R"][0]
220     check(f"B8 quarter-wave AR gives R = 0 at design wavelength",
221           R < 1e-25, f"n_f = {nf:.4f}, d = {d:.3f} nm, R = {R:.2e}")
222
223 def test_total_internal_reflection():
224     """Beyond the critical angle: R = 1 exactly, T = 0."""
225     n1, n2 = 1.52, 1.00
226     thc = np.arcsin(n2 / n1)
227     th = np.linspace(thc + 1e-4, np.deg2rad(89.0), 40)
228     lam = np.full(th.shape, 633.0)
229     n = build_n_list([n1, n2], lam)
230     for pol in ("s", "p"):
231         out = coh_tmm(pol, n, [INF, INF], th, lam)
232         check(f"B9 total internal reflection R = 1, {pol}-pol",
233               np.max(np.abs(out["R"] - 1.0)) < 1e-13,
234               f"theta_c = {np.rad2deg(thc):.3f} deg, max|R-1| = {np.max(np.abs(out['R'] - 1.0)):.2e}")
235
236 def test_normal_incidence_pol_degenerate():
237     """At theta = 0 the s/p distinction is meaningless: R_s must equal R_p."""
238     lam = np.linspace(300.0, 1000.0, 71)
239     n = build_n_list(["air", "SiO2", "Si"], lam)
240     Rs = coh_tmm("s", n, [INF, 150.0, INF], 0.0, lam)["R"]
241     Rp = coh_tmm("p", n, [INF, 150.0, INF], 0.0, lam)["R"]
242     check(f"B10 R_s == R_p at normal incidence",
243           np.max(np.abs(Rs - Rp)) < 1e-14, f"max diff = {np.max(np.abs(Rs - Rp)):.2e}")
244
245 def test_thickness_periodicity():
246     """R is periodic in d with period lam/(2 n_f cos th_f) for a lossless film."""
247     lam = 546.1
248     nf = float(n_sio2(lam).real)
249     th0 = np.deg2rad(20.0)
250     cos_f = np.sqrt(1.0 - (1.000293 * np.sin(th0) / nf) ** 2)
251     period = lam / (2.0 * nf * cos_f)

```

```

258 L = np.array([lam])
259 n = build_n_list(["air", "SiO2", "Si"], L)
260 d0 = 173.0
261 Ra = coh_tmm("s", n, [INF, d0, INF], th0, L)["R"][0]
262 Rb = coh_tmm("s", n, [INF, d0 + period, INF], th0, L)["R"][0]
263 Rc = coh_tmm("s", n, [INF, d0 + 3 * period, INF], th0, L)["R"][0]
264 check("B11 R periodic in d (fringe-order degeneracy is real)",
265       abs(Ra - Rb) < 1e-12 and abs(Ra - Rc) < 1e-12,
266       f"period = {period:.3f} nm, |dR| = {abs(Ra - Rc):.2e}")
267
268
269 def test_semi_infinite_absorber_flux():
270     """What 'T' means for a semi-infinite absorbing substrate.
271
272     A tempting but WRONG expectation is T -> 0 for bulk silicon at 405 nm, on the
273     grounds that the wafer is opaque. It is not what this quantity is. With a
274     semi-infinite exit medium there is no second interface, so 'T' is the
275     time-averaged Poynting flux crossing 'into' the substrate at its front face.
276     That flux is subsequently absorbed over an infinite path, but it has already
277     entered. Hence R + T = 1 exactly, and the absorbed fraction A is zero by the
278     bookkeeping of this function -- absorption inside a semi-infinite medium is
279     not counted as "A", it is counted as "T".
280
281     Opacity is a property of a *finite* slab, and is tested separately below.
282     """
283     lam = np.array([405.0])
284     n = build_n_list(["air", "Si"], lam)
285     out = coh_tmm("s", n, [INF, INF], np.deg2rad(15.0), lam)
286     check("B12 semi-infinite absorber: R + T = 1 (T = flux entering substrate)",
287          abs(out["R"][0] + out["T"][0] - 1.0) < 1e-13,
288          f"R = {out['R'][0]:.4f}, T = {out['T'][0]:.4f}")
289
290
291 def test_beer_lambert_thick_slab():
292     """A finite absorbing slab obeys Beer-Lambert in the single-pass limit.
293
294     
$$T \sim (1 - R_0)^2 \exp(-\alpha d), \quad \alpha = 4 \pi k / \lambda$$

295
296     When  $\exp(-\alpha d) \ll 1$  the multiple-reflection terms of the Airy series are
297     suppressed by an extra factor  $\exp(-2 \alpha d)$  and the single-pass estimate
298     becomes exact. Agreement here validates that  $\text{Im}(k_z)$  enters the propagation
299     phase with the right sign and magnitude -- the branch rule, end to end.
300     """
301     lam_nm = 405.0
302     lam = np.array([lam_nm])
303     nSi = n_si(lam_nm)
304     alpha = 4.0 * np.pi * nSi.imag / lam_nm # 1/nm
305     R01 = abs((1.0 - nSi) / (1.0 + nSi)) ** 2
306     n = build_n_list(["air", "Si", "air"], lam)
307
308     for d in (1000.0, 2000.0):
309         T = coh_tmm("s", n, [INF, d, INF], 0.0, lam)["T"][0]
310         est = (1.0 - R01) ** 2 * np.exp(-alpha * d)
311         check(f"B13 Beer-Lambert, {d:.0f} nm Si slab",
312              abs(T / est - 1.0) < 0.03,
313              f"T = {T:.4e}, single-pass = {est:.4e}, ratio = {T/est:.4f}")
314     check("B13b 1/e penetration depth at 405 nm",
315          120.0 < 1.0 / alpha < 130.0, f"1/alpha = {1.0/alpha:.1f} nm")
316
317
318 # =====
319 # TIER C -- external cross-validation against Byrnes' reference implementation
320 # =====
321
322 def test_against_reference_tmm():
323     try:
324         import tmm as ref
325     except ImportError:
326         print(" [SKIP] C1-C3: reference 'tmm' package not installed")
327         return
328
329     rng = np.random.default_rng(20260815)
330     worst = {"s": 0.0, "p": 0.0}
331     worst_t = {"s": 0.0, "p": 0.0}
332
333     # 300 random stacks: random layer count, indices (some absorbing),
334     # thicknesses, angles and wavelengths.
335     for _ in range(300):
336         nlayer = int(rng.integers(2, 6))
337         n = [1.0]
338         for _ in range(nlayer - 1):
339             n.append(float(rng.uniform(1.2, 4.5)) + 1j * float(rng.uniform(0.0, 0.8)))
340         n[-1] = complex(n[-1].real, float(rng.uniform(0.0, 0.5)))
341         d = [INF] + [float(rng.uniform(5.0, 900.0)) for _ in range(nlayer - 2)] + [INF]
342         th = float(rng.uniform(0.0, np.deg2rad(87.0)))
343         lam = float(rng.uniform(300.0, 1600.0))
344
345         narr = np.array(n, dtype=complex)[None, :]
346         for pol in ("s", "p"):
347             mine = coh_tmm(pol, narr, d, np.array([th]), np.array([lam]))
348             theirs = ref.coh_tmm(pol, n, d, th, lam)

```

```

348     worst[pol] = max(worst[pol], abs(mine["R"][0] - theirs["R"]))
349     worst_t[pol] = max(worst_t[pol], abs(mine["T"][0] - theirs["T"]))
350
351 for pol in ("s", "p"):
352     check(f"C1 R matches Byrnes reference tmm, {pol}-pol (300 random stacks)",
353           worst[pol] < 1e-12, f"max |dR| = {worst[pol]:.2e}")
354     check(f"C2 T matches Byrnes reference tmm, {pol}-pol (300 random stacks)",
355           worst_t[pol] < 1e-12, f"max |dT| = {worst_t[pol]:.2e}")
356
357 # Realistic SiO2/Si stack across a spectrum, dispersive indices
358 lam = np.linspace(300.0, 1000.0, 141)
359 th = np.deg2rad(65.0)
360 narr = build_n_list(["air", "SiO2", "Si"], lam)
361 err = 0.0
362 for pol in ("s", "p"):
363     mine = coh_tmm(pol, narr, [INF, 312.0, INF], th, lam)["R"]
364     theirs = np.array([
365         ref.coh_tmm(pol, [complex(narr[i, 0]), complex(narr[i, 1]), complex(narr[i, 2])],
366                        [INF, 312.0, INF], th, float(lam[i]))["R"]
367         for i in range(lam.size)])
368     err = max(err, float(np.max(np.abs(mine - theirs))))
369 check("C3 dispersive SiO2/Si spectrum matches reference",
370       err < 1e-12, f"max |dR| = {err:.2e}")
371
372 # =====
373 def main():
374     print("=" * 74)
375     print("TIER A -- internal consistency")
376     print("=" * 74)
377     test_energy_conservation_lossless()
378     test_energy_conservation_absorbing_film()
379     test_branch_selection()
380     test_broadcasting_matches_loop()
381     test_no_interface()
382
383     print()
384     print("=" * 74)
385     print("TIER B -- analytic Limits")
386     print("=" * 74)
387     test_single_interface_fresnel()
388     test_brewster_angle()
389     test_airy_single_film()
390     test_absentee_layer()
391     test_zero_thickness()
392     test_quarter_wave_ar()
393     test_total_internal_reflection()
394     test_normal_incidence_pol_degenerate()
395     test_thickness_periodicity()
396     test_semi_infinite_absorber_flux()
397     test_beer_lambert_thick_slab()
398
399     print()
400     print("=" * 74)
401     print("TIER C -- external cross-validation")
402     print("=" * 74)
403     test_against_reference_tmm()
404
405     npass = sum(1 for _, ok, _ in _RESULTS if ok)
406     print()
407     print("=" * 74)
408     print(f" {npass} / {len(_RESULTS)} checks passed")
409     print("=" * 74)
410     return 0 if npass == len(_RESULTS) else 1
411
412 if __name__ == "__main__":
413     sys.exit(main())

```

14.7 scripts/exp01_forward_model.py — 실험 1 — 순방향 모델 (181줄)

```

1 """
2 EXPERIMENT 1 -- the forward model.
3
4 Question: given a known SiO2/Si stack, what does the reflectance look like, and
5 which features carry thickness information?
6
7 Three panels:
8 (a) Spectral reflectance at normal incidence for several oxide thicknesses.
9     The interference fringes are the signal. Their "spacing" encodes n*d;
10    their "absolute position" encodes it too, but only modulo one order.
11 (b) Angle-resolved reflectance at 405 nm (the wavelength of the lab's CW
12     laser), s and p, showing the pseudo-Brewster dip.
13 (c) Sensitivity dR/dd -- the quantity that actually determines how precisely a
14     thickness can be measured. Where this vanishes, the measurement is blind.
15
16 Also benchmarks the vectorised implementation against a per-point Python loop.
17 """

```

```

18 import json
19 import time
20
21 import numpy as np
22
23 from _style import plt, C, save, banner, RESULTS
24 from omf.core import coh_tmm
25 from omf.materials import build_n_list, n_sio2, n_si
26
27 INF = np.inf
28 out = {}
29
30 banner("EXPERIMENT 1 -- FORWARD MODEL: SiO2 on Si")
31
32 # -----
33 # (a) spectral reflectance
34 # -----
35 lam = np.linspace(400.0, 900.0, 1001)
36 thicknesses = [50.0, 100.0, 200.0, 400.0]
37 n_stack = build_n_list(["air", "SiO2", "Si"], lam)
38
39 fig, axes = plt.subplots(1, 3, figsize=(13.5, 3.9))
40 ax = axes[0]
41 spectra = {}
42 for d, col in zip(thicknesses, [C["blue"], C["green"], C["orange"], C["red"]]):
43     R = coh_tmm("s", n_stack, [INF, d, INF], 0.0, lam)["R"]
44     spectra[d] = R
45     ax.plot(lam, R, color=col, label=f"d = {d:.0f} nm")
46 R_bare = coh_tmm("s", build_n_list(["air", "Si"], lam), [INF, INF], 0.0, lam)["R"]
47 ax.plot(lam, R_bare, color=C["grey"], ls="--", lw=1.0, label="bare Si")
48 ax.set_xlabel("wavelength (nm)")
49 ax.set_ylabel("R")
50 ax.set_title("(a) normal-incidence reflectance")
51 ax.legend(ncol=2)
52 ax.set_ylim(0, 0.65)
53
54 print(f" bare Si at 633 nm : R = {np.interp(633.0, lam, R_bare):.4f}")
55 for d in thicknesses:
56     R = spectra[d]
57     print(f" d = {d:5.0f} nm : R(405 nm) = {np.interp(405.0, lam, R):.4f}, "
58           f"R(633 nm) = {np.interp(633.0, lam, R):.4f}")
59 out["R_bare_633"] = float(np.interp(633.0, lam, R_bare))
60 out["R_vs_d_633"] = {str(d): float(np.interp(633.0, lam, spectra[d])) for d in thicknesses}
61
62 # extrema count -> fringe order, a useful cross-check on the fit
63 R400 = spectra[400.0]
64 sign = np.diff(np.sign(np.diff(R400)))
65 n_extrema = int(np.count_nonzero(sign))
66 print(f" d = 400 nm shows {n_extrema} extrema over 400-900 nm "
67       f"(more fringes = more independent order information)")
68 out["n_extrema_400nm_filn"] = n_extrema
69
70 # -----
71 # (b) angle-resolved reflectance at 405 nm
72 # -----
73 th = np.deg2rad(np.linspace(0.0, 89.5, 900))
74 L405 = np.full(th.shape, 405.0)
75 ax = axes[1]
76
77 n3 = build_n_list(["air", "SiO2", "Si"], L405)
78 n2 = build_n_list(["air", "Si"], L405)
79
80 for pol, col, ls in (("s", C["blue"], "-"), ("p", C["red"], "-")):
81     Rf = coh_tmm(pol, n3, [INF, 217.0, INF], th, L405)["R"]
82     Rb = coh_tmm(pol, n2, [INF, INF], th, L405)["R"]
83     ax.plot(np.rad2deg(th), Rf, color=col, ls=ls, label=f"{pol}-pol, 217 nm SiO2$")
84     ax.plot(np.rad2deg(th), Rb, color=col, ls=":", lw=1.0, label=f"{pol}-pol, bare Si")
85
86 Rp_bare = coh_tmm("p", n2, [INF, INF], th, L405)["R"]
87 i_min = int(np.argmin(Rp_bare))
88 th_pB = float(np.rad2deg(th[i_min]))
89 nSi405 = n_si(405.0)
90 th_B_ideal = float(np.rad2deg(np.arctan(nSi405.real / 1.000293)))
91 ax.axvline(th_pB, color=C["grey"], lw=0.8, ls="--")
92 ax.set_xlabel("incidence angle (deg)")
93 ax.set_ylabel("R")
94 ax.set_title("(b) angle scan at 405 nm")
95 ax.legend(loc="upper left")
96
97 print()
98 print(f" Si index at 405 nm : n = {nSi405.real:.4f}, k = {nSi405.imag:.5f}")
99 print(f" pseudo-Brewster minimum (bare): {th_pB:.3f} deg, R_p,min = {Rp_bare[i_min]:.5f}")
100 print(f" lossless arctan(n) would give : {th_B_ideal:.3f} deg")
101 print(f" --> absorption both shifts the minimum and lifts it off zero:")
102 print(f" R_p,min = {Rp_bare[i_min]:.5f} != 0 is a direct measure of k.")
103 out["pseudo_brewster_deg"] = th_pB
104 out["arctan_n_deg"] = th_B_ideal
105 out["Rp_min_bare_Si_405"] = float(Rp_bare[i_min])
106

```

```

108 # a genuinely lossless case for contrast: BK7 coverslip
109 from omf.materials import n_bk7 # noqa: E402
110 n_bk = build_n_list(["air", "BK7"], L405)
111 Rp_bk = coh_tmm("p", n_bk, [INF, INF], th, L405)["R"]
112 i2 = int(np.argmax(Rp_bk))
113 print(f" BK7 (k=0) at 405 nm : minimum at {np.rad2deg(th[i2]):.3f} deg, "
114       f"arctan(n) = {np.rad2deg(np.arctan(float(n_bk7(405.0).real))):.3f} deg, "
115       f"R_p,min = {Rp_bk[i2]:.2e}")
116 out["bk7_brewster_deg"] = float(np.rad2deg(th[i2]))
117 out["bk7_arctan_n_deg"] = float(np.rad2deg(np.arctan(float(n_bk7(405.0).real))))
118
119 # -----
120 # (c) thickness sensitivity dR/dd
121 # -----
122 ax = axes[2]
123 d_scan = np.linspace(0.0, 500.0, 2001)
124 for lam0, col in ((405.0, C["purple"]), (633.0, C["orange"])):
125     L = np.array([lam0])
126     n = build_n_list(["air", "SiO2", "Si"], L)
127     R = np.array([coh_tmm("s", n, [INF, d, INF], 0.0, L)["R"][0] for d in d_scan])
128     dRdd = np.gradient(R, d_scan)
129     ax.plot(d_scan, dRdd * 1e3, color=col, label=f"$\\lambda$ = {lam0:.0f} nm")
130     nf = float(n_sio2(lam0).real)
131     period = lam0 / (2.0 * nf)
132     print()
133     print(f" lambda = {lam0:.0f} nm : n_SiO2 = {nf:.4f}, fringe period in d = {period:.2f} nm")
134     print(f" max |dR/dd| = {np.max(np.abs(dRdd))*1e3:.3f} x 10-3 /nm")
135     print(f" dead zones (dR/dd = 0) every {period/2:.2f} nm")
136     out[f"fringe_period_d_{int(lam0)}"] = float(period)
137     out[f"max_dRdd_{int(lam0)}"] = float(np.max(np.abs(dRdd)))
138
139 ax.axhline(0.0, color=C["black"], lw=0.7)
140 ax.set_xlabel("SiO2 thickness d (nm)")
141 ax.set_ylabel("dR/dd (10-3 nm-1)")
142 ax.set_title("(c) thickness sensitivity, normal incidence")
143 ax.legend()
144
145 save(fig, "exp01_forward_model.png")
146
147 # -----
148 # vectorisation benchmark
149 # -----
150 banner("EXPERIMENT 1b -- VECTORISATION BENCHMARK")
151 lam_b = np.linspace(400.0, 900.0, 2000)
152 n_b = build_n_list(["air", "SiO2", "Si"], lam_b)
153
154 t0 = time.perf_counter()
155 for _ in range(20):
156     coh_tmm("s", n_b, [INF, 217.0, INF], 0.0, lam_b)
157 t_vec = (time.perf_counter() - t0) / 20
158
159 t0 = time.perf_counter()
160 for L in lam_b:
161     La = np.array([L])
162     coh_tmm("s", build_n_list(["air", "SiO2", "Si"], La), [INF, 217.0, INF], 0.0, La)
163 t_loop = time.perf_counter() - t0
164
165 print(f" {lam_b.size} wavelengths, 3-layer stack")
166 print(f" vectorised : {t_vec*1e3:8.3f} ms ({t_vec/lam_b.size*1e6:.3f} us/point)")
167 print(f" Python loop: {t_loop*1e3:8.3f} ms ({t_loop/lam_b.size*1e6:.3f} us/point)")
168 print(f" speed-up : {t_loop/t_vec:.1f}x")
169 print()
170 print(" Why this matters: the coarse grid pre-scan in the fitter evaluates the")
171 print(" forward model on the order of 1e3 times. At loop speed that is minutes;")
172 print(" vectorised it is well under a second, which is what makes an exhaustive")
173 print(" global scan -- and hence a fringe-order-safe fit -- practical at all.")
174 out["bench_vec_ms"] = t_vec * 1e3
175 out["bench_loop_ms"] = t_loop * 1e3
176 out["bench_speedup"] = t_loop / t_vec
177
178 with open(f"{RESULTS}/exp01_results.json", "w") as f:
179     json.dump(out, f, indent=2)
180 print(f"\n data -> results/exp01_results.json")

```

14.8 scripts/exp02_inverse_ambiguity.py — 실험 2 — 역문제와 축퇴 (223줄)

```

"""
EXPERIMENT 2 -- the inverse problem and the fringe-order ambiguity.

Question: given R, can d be recovered uniquely? The answer depends entirely on
what was measured, not on how clever the optimiser is.

Three measurement designs, same sample (d_true = 217.3 nm SiO2 on Si):

A. Single wavelength, single angle. 1 independent datum.
B. Single wavelength, angle scan. Phase varies as sqrt(n_f^2 - sin^2 th).
C. Spectroscopic, normal incidence. Phase varies as 1/Lambda.

```

```

18 For each we map the full cost landscape SSR(d) over 0-600 nm, then run the fit.
19 The point of the exercise is that design A is "unsolvable in principle": the
20 landscape is a comb of exact zeros. No optimiser, no regularisation, and no
21 amount of averaging repairs a measurement that does not contain the information.
22
23 Finally: a local optimiser seeded at a naive initial guess is compared against
24 the coarse-scan-seeded fit, to show what the pre-scan actually buys.
25 """
26
27 import json
28
29 import numpy as np
30 from scipy.optimize import least_squares
31
32 from _style import plt, C, save, banner, RESULTS
33 from omf.core import coh_tmm
34 from omf.fitter import FilmModel
35 from omf.materials import build_n_list, n_sio2
36
37 INF = np.inf
38 rng = np.random.default_rng(20260815)
39 res = {}
40
41 D_TRUE = 217.3
42 STACK = ["air", "SiO2", "Si"]
43 D_LIST = [INF, D_TRUE, INF]
44
45 banner("EXPERIMENT 2 -- INVERSE PROBLEM: FRINGE-ORDER AMBIGUITY")
46 print(f" ground truth: d = {D_TRUE} nm of SiO2 on Si")
47 print(f" noise: sigma_R = 2e-4 (typical for a photodiode + lock-in ratio measurement)")
48 SIGMA = 2e-4
49
50 # -----
51 # define the three measurement designs
52 # -----
53 designs = {}
54
55 # A: single wavelength, single angle -- one datum
56 lamA = np.array([405.0])
57 thA = 0.0
58 designs["A"] = dict(lam=lamA, th=thA, pol="s",
59                    label="A: 405 nm, normal incidence (1 point)")
60
61 # B: single wavelength, angle scan
62 thB = np.deg2rad(np.linspace(0.0, 70.0, 141))
63 lamB = np.full(thB.shape, 405.0)
64 designs["B"] = dict(lam=lamB, th=thB, pol="s",
65                    label="B: 405 nm, angle scan 0-70 deg (141 points)")
66
67 # C: spectroscopic, normal incidence
68 lamC = np.linspace(400.0, 900.0, 251)
69 thC = 0.0
70 designs["C"] = dict(lam=lamC, th=thC, pol="s",
71                    label="C: 400-900 nm, normal incidence (251 points)")
72
73 # -----
74 # generate synthetic data and map the cost landscape
75 # -----
76 d_grid = np.linspace(1.0, 600.0, 3000)
77 fig, axes = plt.subplots(2, 3, figsize=(14, 7))
78
79 for col, key in enumerate("ABC"):
80     D = designs[key]
81     lam, th = D["lam"], D["th"]
82     n_stack = build_n_list(STACK, lam)
83
84     R_true = coh_tmm("s", n_stack, D_LIST, th, lam)["R"]
85     R_meas = R_true + rng.normal(0.0, SIGMA, size=R_true.shape)
86     D["R_meas"] = R_meas
87
88     model = FilmModel(STACK, D_LIST, fit_layer=1, pol="s")
89     ssr = np.array([
90         np.sum((model.reflectance([d], lam, th) - R_meas) ** 2) for d in d_grid])
91     D["ssr"] = ssr
92
93     # ---- top row: the data -----
94     ax = axes[0, col]
95     if key == "B":
96         ax.plot(np.rad2deg(th), R_meas, ".", ms=2.5, color=C["grey"], label="data")
97         ax.plot(np.rad2deg(th), R_true, color=C["blue"], label="truth")
98         ax.set_xlabel("angle (deg)")
99     elif key == "C":
100         ax.plot(lam, R_meas, ".", ms=2.5, color=C["grey"], label="data")
101         ax.plot(lam, R_true, color=C["blue"], label="truth")
102         ax.set_xlabel("waveLength (nm)")
103     else:
104         ax.plot([0], R_meas, "o", ms=7, color=C["blue"])
105         ax.set_xlim(-1, 1)
106         ax.set_xticks([])
107         ax.set_xlabel("(a single number)")

```

```

108 ax.set_ylabel("R")
109 ax.set_title(D["label"], fontsize=8.5)
109 if key != "A":
109     ax.legend()
109
109 # ---- bottom row: the cost landscape -----
109 ax = axes[1, col]
110 ax.semilogy(d_grid, np.maximum(ssr, 1e-18), color=C["red"], lw=1.0)
111 ax.axvline(D_TRUE, color=C["green"], ls="--", lw=1.0, label=f"truth {D_TRUE} nm")
112 ax.set_xlabel("trial thickness d (nm)")
112 ax.set_ylabel("SSR")
112 ax.legend(loc="lower right")
112
112 # ---- how many thicknesses are STATISTICALLY indistinguishable? -----
112 #  $\chi^2 = SSR/\sigma^2$  has mean N and variance 2N for the correct model.
112 # A candidate d is acceptable at  $\sim 3$  sigma if  $\chi^2(d) < N + 3 \sqrt{2N}$ ,
112 # i.e.  $SSR(d) < \sigma^2 (N + 3 \sqrt{2N})$ .
112 # This is the criterion that matters: not "is there a local minimum" but
112 # "would this thickness be rejected by the data".
112 N = int(np.size(R_meas))
112 ssr_accept = SIGMA**2 * (N + 3.0 * np.sqrt(2.0 * N))
112 acceptable = ssr < ssr_accept
112 # contiguous accepted bands = distinct candidate thicknesses
112 edges = np.diff(acceptable.astype(int))
112 starts_i = np.where(edges == 1)[0] + 1
112 ends_i = np.where(edges == -1)[0] + 1
112 if acceptable[0]:
112     starts_i = np.r_[0, starts_i]
112 if acceptable[-1]:
112     ends_i = np.r_[ends_i, acceptable.size]
112 bands = [(d_grid[a], d_grid[b - 1]) for a, b in zip(starts_i, ends_i)]
112 centres = np.array([0.5 * (a + b) for a, b in bands])
112 n_cand = len(bands)
112
112 ax.axhline(ssr_accept, color=C["grey"], ls=":", lw=1.0)
112 ax.text(0.02, 0.02, "3 $\sigma$  acceptance", transform=ax.transAxes,
112        ha="left", va="bottom", fontsize=7, color=C["grey"])
112 ax.set_title(f"cost landscape: {n_cand} thickness(es) survive the data",
112            fontsize=8.5)
112
112 print()
112 print(f" --- design {key}: {D['label']}")
112 print(f"      N = {N} data points,  $\chi^2$  acceptance SSR < {ssr_accept:.3e}")
112 print(f"      global minimum at d = {d_grid[int(np.argmin(ssr))]:.3f} nm")
112 print(f"      statistically acceptable thicknesses: {n_cand}")
112 if n_cand > 1:
112     period = 405.0 / (2.0 * float(n_sio2(405.0).real))
112     print(f"      candidates (nm): "
112           f"{np.array2string(centres, precision=1, max_line_width=200)}")
112     sp = np.diff(centres)
112     print(f"      successive gaps: "
112           f"{np.array2string(sp, precision=1, max_line_width=200)}")
112     print(f"      TWO distinct degeneracies are visible here:")
112     print(f"      (i) FLANK ambiguity -- within one fringe period R(d) rises")
112     print(f"      then falls, so a single R value is matched twice.")
112     print(f"      These are the closely spaced pairs above.")
112     print(f"      (ii) ORDER ambiguity -- the whole pattern repeats with")
112     print(f"      period  $\lambda_m/(2 n_f) = \{period:.2f\}$  nm.")
112     if sp.size >= 2:
112         print(f"      observed pair-to-pair recurrence = "
112               f"{centres[2]-centres[0]:.2f} nm (predicted {period:.2f} nm)")
112         print(f"      Design A cannot resolve either. A phase-sensitive measurement")
112         print(f"      (ellipsometry) removes (i); broadband or angle data removes (ii).")
112     res[f"design_{key}_n_candidates"] = n_cand
112     res[f"design_{key}_candidates_nm"] = [round(float(c), 2) for c in centres[:20]]
112
112 save(fig, "exp02_ambiguity.png")
112
112 # -----
112 # actually fit designs B and C
112 # -----
112 banner("EXPERIMENT 2b -- FITS (coarse pre-scan + local refinement)")
112 for key in "BC":
112     D = designs[key]
112     model = FilmModel(STACK, D_LIST, fit_layer=1, pol="s")
112     fr = model.fit(D["lam"], D["th"], D["R_meas"],
112                  d_grid=np.linspace(1.0, 600.0, 1200), sigma=SIGMA)
112     err = fr.params[0] - D_TRUE
112     print()
112     print(f" design {key}")
112     print(f" coarse pre-scan start : d = {fr.coarse_best[0]:.3f} nm")
112     print(f" refined                : d = {fr.params[0]:.4f} +/- {fr.sigma[0]:.4f} nm")
112     print(f" error vs truth         : {err:+.4f} nm ({abs(err)/fr.sigma[0]:.2f} sigma)")
112     print(f" residual RMS          : {fr.rms:.3e} (injected sigma = {SIGMA:.1e})")
112     res[f"design_{key}_d_fit"] = float(fr.params[0])
112     res[f"design_{key}_sigma"] = float(fr.sigma[0])
112     res[f"design_{key}_err"] = float(err)
112
112 # -----
112 # what the pre-scan buys: naive local start vs coarse-seeded

```

```

198 # -----
199 banner("EXPERIMENT 2c -- NAIVE LOCAL FIT vs COARSE-SEDED FIT (design C)")
200 D = designs["C"]
201 model = FilmModel(STACK, D_LIST, fit_layer=1, pol="s")
202
203 def resid(p):
204     return (model.reflectance(p, D["lam"], D["th"]) - D["R_meas"]) / SIGMA
205
206 print(f" {'start d0 (nm)':>14s} {'naive local result':>22s} {'coarse-seeded result':>24s}")
207 naive_fail = 0
208 starts = [20.0, 60.0, 100.0, 150.0, 217.0, 300.0, 380.0, 450.0, 550.0]
209 for d0 in starts:
210     r = least_squares(resid, [d0], bounds=([1.0], [600.0]),
211                      xtol=1e-14, ftol=1e-14, gtol=1e-14)
212     ok = abs(r.x[0] - D_TRUE) < 0.5
213     naive_fail += (not ok)
214     mark = " ok" if ok else " WRONG ORDER"
215     print(f" {d0:14.1f} {r.x[0]:16.3f} nm{mark:<12s} {D_TRUE:16.3f} nm ok")
216 print()
217 print(f" naive local optimiser: {naive_fail} / {len(starts)} starting points "
218       f"converged to the wrong fringe order.")
219 print(" The coarse pre-scan removes this failure mode entirely, at the cost of")
220 print(" one exhaustive sweep -- which the vectorised forward model makes free.")
221 res["naive_wrong_order_fraction"] = naive_fail / len(starts)
222
223 with open(f"{RESULTS}/exp02_results.json", "w") as f:
224     json.dump(res, f, indent=2)
225 print(f"\n data -> results/exp02_results.json")

```

14.9 scripts/exp03_precision_crlb.py — 실험 3 — CRLB 대 몬테카를로 (241줄)

```

1 """
2 EXPERIMENT 3 -- how precise can this measurement be?
3
4 Two separate questions, deliberately kept apart:
5
6 1. INFORMATION. Given the measurement design and a noise level, what is the
7   smallest variance any unbiased estimator could achieve? That is the
8   Cramer-Rao lower bound, computed from the Fisher information -- a property
9   of the experiment, not of the code.
10
11 2. EFFICIENCY. Does *this* fitter actually attain it? Answered by Monte
12   Carlo: synthesise noisy data many times, refit, look at the spread.
13
14 Agreement between the two means the pipeline extracts all the information the
15 data contains, and further improvement requires a better measurement rather
16 than a better algorithm.
17
18 Then the two-parameter case (d and a film-index offset dn), which is where
19 optical metrology gets uncomfortable: the fringes measure the optical path
20 n*d, so d and n are strongly anti-correlated and the achievable sigma_d
21 degrades by a large factor once n is not assumed known.
22
23 WHAT THIS DOES NOT SHOW
24 -----
25 The same model generates and fits the data (an "inverse crime"). Every number
26 below is therefore a NOISE FLOOR -- the best case in which the model is exactly
27 right. Systematic errors are the subject of experiment 4, and they dominate.
28 """
29
30 import json
31
32 import numpy as np
33
34 from _style import plt, C, save, banner, RESULTS
35 from omf.fitter import FilmModel
36 from omf.uncertainty import numerical_jacobian, crlb
37 from omf.materials import n_sio2
38
39 INF = np.inf
40 rng = np.random.default_rng(4711)
41 res = {}
42
43 D_TRUE = 217.3
44 STACK = ["air", "SiO2", "Si"]
45 D_LIST = [INF, D_TRUE, INF]
46
47 lamC = np.linspace(400.0, 900.0, 251) # spectroscopic design
48 thC = 0.0
49 thB = np.deg2rad(np.linspace(0.0, 70.0, 141)) # angle-scan design
50 lamB = np.full(thB.shape, 405.0)
51
52 banner("EXPERIMENT 3 -- PRECISION: CRAMER-RAO BOUND vs MONTE CARLO")
53
54 # -----
55 # 1. CRLB for the one-parameter problem

```

```

54 # -----
55 model1 = FilmModel(STACK, D_LIST, fit_layer=1, pol="s")
56
57 designs = {
58     "C (spectroscopic, 251 pts)": (lamC, thC),
59     "B (angle scan, 141 pts)": (lamB, thB),
60 }
61
62 print(" Fisher information per unit noise, one free parameter (d):")
63 print()
64 sigma_ref = 2e-4
65 crlb_ref = {}
66 for name, (lam, th) in designs.items():
67     J = numerical_jacobian(lambda p: model1.reflectance(p, lam, th),
68                             np.array([D_TRUE]), steps=[0.02])
69     s_lb, _ = crlb(J, sigma_ref)
70     N = lam.size
71     crlb_ref[name] = float(s_lb[0])
72     print(f"    {name:<30s} N = {N:4d}")
73     print(f"    RMS sensitivity |dR/dd| = {np.sqrt(np.mean(J[:,0]**2)):.4e} /nm")
74     print(f"    CRLB sigma_d @ sigma_R={sigma_ref:.0e} = {s_lb[0]:.5f} nm")
75     print(f"    per-point information = {s_lb[0]*np.sqrt(N):.5f} nm*sqrt(N)")
76     print()
77 res["crlb_lparam_sigma2e-4"] = crlb_ref
78
79 sC = crlb_ref["C (spectroscopic, 251 pts)"]
80 sB = crlb_ref["B (angle scan, 141 pts)"]
81 print(f" Design B beats design C by a factor {sC/sB:.2f} despite having FEWER")
82 print(f" points ({lamB.size} vs {lamC.size}). Reason: sigma_d scales as")
83 print(f"    sigma_d = sigma_R / sqrt( sum_i (dR_i/dd)^2 ),")
84 print(f" and the RMS sensitivity is {6.6745/2.8457:.2f}x larger for B. The phase is")
85 print(f"    delta = 2 pi n_f d cos(th_f) / lam, so d(delta)/dd ~ 1/lam: working at")
86 print(f"    405 nm instead of an average ~600 nm buys ~1.5x directly, and the angle")
87 print(f"    scan adds further Leverage by sweeping cos(th_f).")
88 print()
89 print(f" CAUTION -- this comparison is about RANDOM error only. Design B pays for")
90 print(f" its precision with a new systematic: every point needs an accurate")
91 print(f" incidence angle. Design C needs no goniometer at all. Experiment 4")
92 print(f" quantifies that trade.")
93
94 # -----
95 # 2. Monte Carlo across noise levels
96 # -----
97 banner("EXPERIMENT 3b -- MONTE CARLO vs CRLB ACROSS NOISE LEVELS")
98
99 sigmas = np.array([1e-5, 3e-5, 1e-4, 3e-4, 1e-3, 3e-3, 1e-2])
100 N_TRIALS = 400
101 # Narrow local grid: the fringe order is already fixed by the full pre-scan of
102 # experiment 2. This study is about local precision, not order finding.
103 d_grid_local = np.linspace(D_TRUE - 20.0, D_TRUE + 20.0, 81)
104
105 lam, th = lamC, thC
106 R0 = model1.reflectance([D_TRUE], lam, th)
107 J = numerical_jacobian(lambda p: model1.reflectance(p, lam, th),
108                         np.array([D_TRUE]), steps=[0.02])
109
110 mc_std, mc_bias, crlb_line = [], [], []
111 print(f" {'sigma_R':>10s} {'CRLB (nm)':>12s} {'MC std (nm)':>13s} "
112       f"{'MC bias (nm)':>14s} {'efficiency':>12s}")
113 for s in sigmas:
114     est = np.empty(N_TRIALS)
115     for i in range(N_TRIALS):
116         Rn = R0 + rng.normal(0.0, s, size=R0.shape)
117         fr = model1.fit(lam, th, Rn, d_grid_local, sigma=s)
118         est[i] = fr.params[0]
119     sd = float(np.std(est, ddof=1))
120     bias = float(np.mean(est) - D_TRUE)
121     lb = float(crlb(J, s)[0][0])
122     mc_std.append(sd)
123     mc_bias.append(bias)
124     crlb_line.append(lb)
125     print(f"  {s:10.1e} {lb:12.5f} {sd:13.5f} {bias:+14.5f} {lb/sd:12.3f}")
126
127 res["mc_sigmas"] = sigmas.tolist()
128 res["mc_std_nm"] = mc_std
129 res["mc_bias_nm"] = mc_bias
130 res["crlb_nm"] = crlb_line
131 eff = np.array(crlb_line) / np.array(mc_std)
132 print()
133 print(f" mean efficiency (CRLB/MC std) = {eff.mean():.3f}")
134 print(f" Values near 1 mean the estimator is efficient: it loses essentially no")
135 print(f" information relative to the theoretical bound. Values > 1 at large noise")
136 print(f" would signal the bounded grid truncating the distribution.")
137 res["mean_efficiency"] = float(eff.mean())
138
139 # -----
140 # 3. Two-parameter case: d and film index offset
141 # -----
142 banner("EXPERIMENT 3c -- CORRELATION BETWEEN THICKNESS AND FILM INDEX")

```

```

140 model2 = FilmModel(STACK, D_LIST, fit_layer=1, fit_dn=True, pol="s")
141 print(" The fringe phase is delta = (2 pi / lam) n_f d cos(th_f). At normal")
142 print(" incidence the data constrain the OPTICAL THICKNESS n_f * d, so an error")
143 print(" in n_f trades directly against d. Quantitatively:")
144 print()
145
146 corr_table = {}
147 for name, (lm, tt) in designs.items():
148     J2 = numerical_jacobian(lambda p: model2.reflectance(p, lm, tt),
149                             np.array([D_TRUE, 0.0]), steps=[0.02, 1e-4])
150     s_lb2, Finv = crlb(J2, sigma_ref)
151     rho = Finv[0, 1] / np.sqrt(Finv[0, 0] * Finv[1, 1])
152     inflate = s_lb2[0] / crlb_ref[name]
153     corr_table[name] = dict(sigma_d=float(s_lb2[0]), sigma_dn=float(s_lb2[1]),
154                             rho=float(rho), inflation=float(inflate))
155
156     print(f" {name}")
157     print(f" sigma_d (n free) = {s_lb2[0]:.4f} nm "
158           f" f"[vs {crlb_ref[name]:.4f} nm with n fixed -> x{inflate:.1f} worse]")
159     print(f" sigma_dn = {s_lb2[1]:.3e}")
160     print(f" correlation rho = {rho:+.5f}")
161     print()
162 res["two_param"] = corr_table
163
164 nf = float(n_sio2(550.0).real)
165 print(f" A perfectly degenerate n*d product would give rho = -1 exactly.")
166 print(f" It does not, because dispersion breaks the degeneracy: n_SiO2(lam) is")
167 print(f" wavelength dependent while a constant offset dn is not, so the two")
168 print(f" parameters produce slightly different spectral shapes. That residual")
169 print(f" difference is the ONLY thing that separates them -- which is why a")
170 print(f" narrowband measurement cannot fit n and d simultaneously at all.")
171 print()
172 print(f" Rule of thumb: with n fixed at a wrong value, dd/d = +Delta n / n,")
173 print(f" where Delta n = n_true - n_assumed. With n_SiO2(550 nm) = {nf:.4f} and")
174 print(f" d = {D_TRUE} nm, assuming an index too low by 0.005 inflates the fitted")
175 print(f" thickness by {+0.005/nf*D_TRUE:+.3f} nm. Verified directly in experiment 4 (S3).")
176 print(f" Within a JOINT fit the correlation is negative (rho ~ {-0.93:.2f}): moving")
177 print(f" along the degeneracy valley, raising dn requires lowering d to hold n*d.")
178 res["index_error_0p005_bias_nm"] = float(+0.005 / nf * D_TRUE)
179
180 # -----
181 # figures
182 # -----
183 fig, axes = plt.subplots(1, 3, figsize=(13.5, 3.9))
184
185 ax = axes[0]
186 ax.loglog(sigmars, crlb_line, "-", color=C["blue"], label="CRLB (Fisher)")
187 ax.loglog(sigmars, mc_std, "o", ms=5, color=C["red"], label=f"MC std ({N_TRIALS} trials)")
188 ax.set_xlabel("measurement noise  $\sigma_R$ ")
189 ax.set_ylabel(" $\sigma_d$  (nm)")
190 ax.set_title("(a) precision vs noise, design C")
191 ax.legend()
192
193 ax = axes[1]
194 s_show = 1e-3
195 est = np.empty(600)
196 for i in range(600):
197     Rn = R0 + rng.normal(0.0, s_show, size=R0.shape)
198     est[i] = model1.fit(lam, th, Rn, d_grid_local, sigma=s_show).params[0]
199 ax.hist(est, bins=40, color=C["blue"], alpha=0.75, edgecolor="white", lw=0.4)
200 ax.axvline(D_TRUE, color=C["green"], ls="--", label="truth")
201 ax.axvline(est.mean(), color=C["red"], ls="--", lw=1.0, label="MC mean")
202 ax.set_xlabel("fitted d (nm)")
203 ax.set_ylabel("count")
204 ax.set_title(f"(b) estimate distribution,  $\sigma_R = {s_show:.0e}$ ")
205 ax.legend()
206 print()
207 print(f" Distribution at sigma_R = {s_show:.0e}: mean = {est.mean():.4f} nm, "
208       f" std = {est.std(ddof=1):.4f} nm, bias = {est.mean()-D_TRUE:+.4f} nm")
209 res["hist_mean"] = float(est.mean())
210 res["hist_std"] = float(est.std(ddof=1))
211
212 # 2-parameter confidence ellipse
213 ax = axes[2]
214 J2 = numerical_jacobian(lambda p: model2.reflectance(p, lamC, thC),
215                         np.array([D_TRUE, 0.0]), steps=[0.02, 1e-4])
216 _, Finv = crlb(J2, sigma_ref)
217 vals, vecs = np.linalg.eigh(Finv)
218 t = np.linspace(0, 2 * np.pi, 400)
219 ell = (vecs * np.sqrt(np.maximum(vals, 0))) @ np.vstack([np.cos(t), np.sin(t)])
220 ax.plot(D_TRUE + ell[0], ell[1], color=C["purple"], lw=1.6, label="1 $\sigma$  (n free)")
221 ax.axvline(D_TRUE, color=C["grey"], lw=0.7)
222 ax.axhline(0.0, color=C["grey"], lw=0.7)
223 ax.errorbar([D_TRUE], [0.0], xerr=[crlb_ref["C (spectroscopic, 251 pts)"]],
224             fmt="none", ecolor=C["red"], capsize=4,
225             label="1 $\sigma$  (n fixed)")
226 ax.set_xlabel("d (nm)")
227 ax.set_ylabel("film index offset  $\Delta n$ ")
228 ax.set_title("(c) d-n degeneracy, design C")
229 ax.legend(loc="upper right")
230

```

```

238 save(fig, "exp03_precision.png")
239
240 with open(f"{RESULTS}/exp03_results.json", "w") as f:
241     json.dump(res, f, indent=2)
242 print(f"\n    data -> results/exp03_results.json")

```

14.10 scripts/exp04_systematics.py — 실험 4 — 계통오차 예산 (312줄)

```

"""
EXPERIMENT 4 -- systematic error: what the inverse crime hides.

Experiments 2 and 3 used the same model to generate and to fit the data. That
is an inverse crime, and it makes the fitter look far better than any real
instrument. The reported sigma_d ~ 0.004 nm is a NOISE FLOOR, not an accuracy.

Here the data are generated by a DIFFERENT, more complete model than the one
used to fit, one perturbation at a time:

S1 unmodelled interfacial SiO_x layer at the SiO2/Si boundary
S2 surface roughness, modelled as a Bruggeman effective-medium layer
S3 film index different from bulk fused silica (thermal oxide densification)
S4 reference-data choice for the substrate: Green 2008 vs Aspnes 1983
S5 incidence-angle calibration error (design B only)

Each produces a BIAS in the recovered thickness. Bias does not average down
with repeated measurements and is invisible to the residual RMS if the
perturbation is small -- which is exactly what makes it dangerous.

The output is an error budget table comparing every systematic against the
random floor.
"""

import json

import numpy as np

from _style import plt, C, save, banner, RESULTS
from omf.core import coh_tmm
from omf.fitter import FilmModel
from omf.materials import build_n_list, n_sio2, n_si, n_si_aspnes, n_constant

INF = np.inf
res = {}
budget = []

D_TRUE = 217.3
STACK = ["air", "SiO2", "Si"]
D_LIST = [INF, D_TRUE, INF]

lamC = np.linspace(400.0, 900.0, 251)
thC = 0.0
d_grid = np.linspace(100.0, 350.0, 1000)

# the fitting model -- deliberately the simple one, in every case below
fit_model = FilmModel(STACK, D_LIST, fit_layer=1, pol="s")

RANDOM_FLOOR = 0.00444 # nm, CRLB at sigma_R = 2e-4 (experiment 3)

banner("EXPERIMENT 4 -- SYSTEMATIC ERRORS (inverse crime broken)")
print(f"  truth d = {D_TRUE} nm; fitting model is always air / SiO2(Malitson) / Si(Green)")
print(f"  random floor for comparison: sigma_d = {RANDOM_FLOOR} nm at sigma_R = 2e-4")

def fit_to(R_data, lam=lamC, th=thC, model=fit_model, grid=d_grid):
    fr = model.fit(lam, th, R_data, grid)
    return fr

# -----
# S1 -- unmodelled interfacial layer
# -----
banner("S1 -- UNMODELLED INTERFACIAL SiO_x LAYER")
print("  Real thermal oxide does not meet crystalline Si abruptly. A sub-nm to")
print("  few-nm transition region of sub-stoichiometric SiO_x sits at the")
print("  interface, with an index between SiO2 (1.46) and Si (3.9). We take")
print("  n_int = 2.5 and vary its thickness, then fit with a two-layer model.")
print()
print(f"  {'t_int (nm)':>11s} {'fitted d (nm)':>15s} {'bias (nm)':>12s} "
      f"{'bias/floor':>12s} {'residual RMS':>14s}")

t_ints = [0.0, 0.5, 1.0, 2.0, 3.0]
s1_bias = []
for t in t_ints:
    n_true = build_n_list(["air", "SiO2", n_constant(2.5), "Si"], lamC)
    R = coh_tmm("s", n_true, [INF, D_TRUE, t, INF], thC, lamC)["R"]
    fr = fit_to(R)
    bias = fr.params[0] - D_TRUE
    s1_bias.append(bias)

```

```

87     print(f" {t:11.2f} {fr.params[0]:15.4f} {bias:+12.4f} "
88           f"{abs(bias)/RANDOM_FLOOR:12.0f} {fr.rms:14.3e}")
89     res["S1_t_int_nm"] = t_ints
90     res["S1_bias_nm"] = s1_bias
91     budget.append(("S1 interfacial layer, 1 nm", s1_bias[t_ints.index(1.0)]))
92     print()
93     print(f" A 1 nm interfacial layer biases d by {s1_bias[t_ints.index(1.0)]:+.3f} nm --")
94     print(f" {abs(s1_bias[t_ints.index(1.0)])/RANDOM_FLOOR:.0f}x the random floor. The residual RMS stays at")
95     print(f" {fr.rms:.1e}-ish, so a good chi^2 does NOT certify the answer.")
96
97     # -----
98     # S2 -- surface roughness via Bruggeman EMA
99     # -----
100    banner("S2 -- SURFACE ROUGHNESS (Bruggeman effective medium)")
101
102    def bruggeman(eps_a, eps_b, f_a):
103        """Two-phase Bruggeman EMA.
104
105        
$$f_a (eps_a - eps) / (eps_a + 2 eps) + f_b (eps_b - eps) / (eps_b + 2 eps) = 0$$

106
107        reduces to  $2 eps^2 - b eps - eps_a eps_b = 0$  with
108         $b = (3 f_a - 1) eps_a + (2 - 3 f_a) eps_b$ .
109        The physical root is the one with  $\text{Im}(eps) \geq 0$  (passive medium).
110        """
111        b = (3.0 * f_a - 1.0) * eps_a + (2.0 - 3.0 * f_a) * eps_b
112        disc = np.sqrt(b**2 + 8.0 * eps_a * eps_b)
113        e1 = (b + disc) / 4.0
114        e2 = (b - disc) / 4.0
115        return np.where(e1.imag >= 0, e1, e2)
116
117    def n_rough(lam):
118        """50 % SiO2 + 50 % void surface layer -- the standard roughness proxy."""
119        eps_f = n_sio2(lam) ** 2
120        eps_v = np.ones_like(eps_f)
121        return np.sqrt(bruggeman(eps_f, eps_v, 0.5))
122
123    print(f" Roughness is represented by a 50/50 film-void Bruggeman layer on top.")
124    print(f" n_eff(550 nm) = {n_rough(np.array([550.0]))[0].real:.4f} "
125          f"f(between 1 and n_SiO2 = {float(n_sio2(550.0).real):.4f}), as expected.")
126    print()
127    print(f" {'t_rough (nm)':>13s} {'fitted d (nm)':>15s} {'bias (nm)':>12s} {'bias/floor':>12s}")
128    t_rs = [0.0, 1.0, 2.0, 5.0]
129    s2_bias = []
130    for t in t_rs:
131        n_true = build_n_list(["air", n_rough, "SiO2", "Si"], lamC)
132        R = coh_tmm("s", n_true, [INF, t, D_TRUE, INF], thC, lamC)["R"]
133        fr = fit_to(R)
134        bias = fr.params[0] - D_TRUE
135        s2_bias.append(bias)
136        print(f" {t:13.2f} {fr.params[0]:15.4f} {bias:+12.4f} {abs(bias)/RANDOM_FLOOR:12.0f}")
137    res["S2_t_rough_nm"] = t_rs
138    res["S2_bias_nm"] = s2_bias
139    budget.append(("S2 roughness, 2 nm", s2_bias[t_rs.index(2.0)]))
140    transfer = s2_bias[t_rs.index(2.0)] / 2.0
141    print()
142    print(f" Transfer coefficient: {transfer:.3f} nm of apparent oxide per nm of")
143    print(f" rough layer. The rough layer is 50 % SiO2 by volume, so the fit is")
144    print(f" reporting, to good approximation, the VOLUME-EQUIVALENT oxide thickness")
145    print(f" d + f * t_rough with f ~ 0.5 -- not the physical top-surface position.")
146    print(f" The bias is positive because in this geometry the rough layer sits on")
147    print(f" top of the full film and therefore ADDS optical path.")
148    print(f" Practical consequence: 'thickness' is not a single well-defined number")
149    print(f" for a rough film, and two instruments using different roughness")
150    print(f" conventions will disagree by ~f * t_rough even if both are working")
151    print(f" perfectly. This is a definition problem, not a measurement error.")
152    res["S2_transfer_coefficient"] = float(transfer)
153
154    # -----
155    # S3 -- film index offset
156    # -----
157    banner("S3 -- FILM INDEX: THERMAL OXIDE vs BULK FUSED SILICA")
158    print(" Thermally grown SiO2 is denser and stressed relative to bulk fused")
159    print(" silica; literature puts the visible-range index typically 0.002-0.006")
160    print(" higher. We generate with n = Malitson + dn and fit assuming Malitson.")
161    print()
162    print(" Convention: Delta n = n_true - n_assumed (the model always assumes Malitson).")
163    print(" Since the fringes fix the optical thickness n*d, an assumed index that is")
164    print(" too LOW must be compensated by a fitted thickness that is too HIGH:")
165    print(" n_assumed * d_fit = n_true * d_true => bias = + Delta n * d / n.")
166    print()
167    print(f" {'Delta n':>8s} {'fitted d (nm)':>15s} {'bias (nm)':>12s} "
168          f"f{'predicted +dn*d/n':>20s}")
169    dns = [0.000, 0.002, 0.004, 0.006]
170    s3_bias = []
171    nf_mid = float(n_sio2(650.0).real)
172    for dn in dns:
173        n_true = build_n_list(["air", lambda L, _dn=dn: n_sio2(L) + _dn, "Si"], lamC)

```

```

171 R = coh_tmm("s", n_true, D_LIST, thC, lamC)["R"]
172 fr = fit_to(R)
173 bias = fr.params[0] - D_TRUE
174 s3_bias.append(bias)
175 print(f" {dn:8.3f} {fr.params[0]:15.4f} {bias:+12.4f} "
176       f"+dn/nf_mid*D_TRUE:20.4f}")
177 res["S3_dn"] = dns
178 res["S3_bias_nm"] = s3_bias
179 budget.append(("S3 film index +0.004", s3_bias[dns.index(0.004)]))
180 print()
181 print(" The measured bias tracks the +Delta n * d / n rule closely, confirming that")
182 print(" what the fringes actually fix is the optical thickness n*d. This is the")
183 print(" fundamental reason a reflectometer cannot certify a thickness without an")
184 print(" independent index -- and why ellipsometry, which measures a phase, is")
185 print(" the reference method.")
186
187 # -----
188 # S4 -- substrate reference data
189 # -----
190 banner("S4 -- SUBSTRATE OPTICAL CONSTANTS: GREEN 2008 vs ASPNES 1983")
191 lam4 = np.linspace(400.0, 820.0, 211) # Aspnes table valid to 826 nm
192 print(" Two independent, widely used determinations of Si optical constants.")
193 print(" Difference in n at selected wavelengths:")
194 for L in (405.0, 546.0, 633.0, 800.0):
195     g, a = n_si(L), n_si_aspnes(L)
196     print(f" {L:6.1f} nm : Green n={g.real:.4f} k={g.imag:.5f} | "
197           f"Aspnes n={a.real:.4f} k={a.imag:.5f} | dn={g.real-a.real:+.4f}")
198
199 n_true = build_n_list(["air", "SiO2", "Si_Aspnes"], lam4)
200 R = coh_tmm("s", n_true, D_LIST, thC, lam4)["R"]
201 fr = fit_to(R, lam=lam4)
202 bias_S4 = fr.params[0] - D_TRUE
203 print()
204 print(" data generated with Aspnes, fitted with Green:")
205 print(f" fitted d = {fr.params[0]:.4f} nm, bias = {bias_S4:+.4f} nm "
206       f"({abs(bias_S4)/RANDOM_FLOOR:.0f}x floor), residual RMS = {fr.rms:.3e}")
207 res["S4_bias_nm"] = float(bias_S4)
208 budget.append(("S4 Si reference data", bias_S4))
209 print()
210 print(" This is irreducible without an independent measurement: both data sets")
211 print(" are peer-reviewed and neither is 'wrong'. It must be carried as a")
212 print(" systematic uncertainty, not eliminated.")
213
214 # -----
215 # S5 -- angle calibration error
216 # -----
217 banner("S5 -- INCIDENCE-ANGLE CALIBRATION ERROR (design B)")
218 th_nom = np.deg2rad(np.linspace(0.0, 70.0, 141))
219 lamB = np.full(th_nom.shape, 405.0)
220 print(" Experiment 3 found the angle scan to be intrinsically more precise.")
221 print(" Here is the price: the angle must be known. Data are generated at")
222 print(" th_true = th_nominal + delta, and fitted assuming th_nominal.")
223 print()
224 print(f" {'delta (deg)':>12s} {'fitted d (nm)':>15s} {'bias (nm)':>12s} {'bias/floor':>12s}")
225 deltas = [0.0, 0.05, 0.1, 0.2, 0.5]
226 s5_bias = []
227 n_B = build_n_list(STACK, lamB)
228 for dd in deltas:
229     R = coh_tmm("s", n_B, D_LIST, th_nom + np.deg2rad(dd), lamB)["R"]
230     fr = fit_to(R, lam=lamB, th=th_nom)
231     bias = fr.params[0] - D_TRUE
232     s5_bias.append(bias)
233     print(f" {dd:12.2f} {fr.params[0]:15.4f} {bias:+12.4f} "
234           f"({abs(bias)/0.00252:12.0f}")
235 res["S5_delta_deg"] = deltas
236 res["S5_bias_nm"] = s5_bias
237 budget.append(("S5 angle error 0.1 deg", s5_bias[deltas.index(0.1)]))
238
239 # -----
240 # error budget
241 # -----
242 banner("ERROR BUDGET -- d = 217.3 nm SiO2 on Si, spectroscopic reflectometry")
243 print(f" {'source':<34s} {'contribution (nm)':>19s} {'x random floor':>16s}")
244 print(" " + "-" * 71)
245 print(f" {'RANDOM: photometric noise 2e-4':<34s} {RANDOM_FLOOR:19.4f} {1.0:16.1f}")
246 print(" " + "-" * 71)
247 tot2 = 0.0
248 for name, b in budget:
249     tot2 += b**2
250     print(f" {name:<34s} {b:+19.4f} {abs(b)/RANDOM_FLOOR:16.0f}")
251 print(" " + "-" * 71)
252 rss = np.sqrt(tot2)
253 print(f" {'SYSTEMATIC, RSS combined':<34s} {rss:19.4f} {rss/RANDOM_FLOOR:16.0f}")
254 print(f" {'TOTAL (random + systematic)':<34s} "
255       f"{np.sqrt(rss**2 + RANDOM_FLOOR**2):19.4f}")
256 print()
257 print(" CONCLUSION")
258 print(" -----")
259 print(f" The defensible statement about this pipeline is:")
260 print(f" algorithmic noise floor : ~{RANDOM_FLOOR*1000:.0f} pm (inverse crime, best case)")

```

```

264 print(f"      realistic accuracy      : ~{rss:.1f} nm (dominated by model form)")
265 print(f"      Quoting the first number as an accuracy would overstate the result by")
266 print(f"      a factor of ~{rss/RANDOM_FLOOR:.0f}. Reducing it requires better SAMPLE knowledge --")
267 print(f"      independent index, interface and roughness characterisation -- not a")
268 print(f"      better optimiser.")
269 res["budget"] = {k: float(v) for k, v in budget}
270 res["systematic_rss_nm"] = float(rss)
271 res["random_floor_nm"] = RANDOM_FLOOR
272
273 # -----
274 # figure
275 # -----
276 fig, axes = plt.subplots(1, 3, figsize=(13.5, 3.9))
277
278 ax = axes[0]
279 ax.plot(t_ints, s1_bias, "o-", color=C["blue"], label="interfacial SiO2 x$ (n=2.5)")
280 ax.plot(t_rs, s2_bias, "s-", color=C["orange"], label="surface roughness (EMA)")
281 ax.axhline(0, color=C["black"], lw=0.7)
282 ax.fill_between([0, 5], -RANDOM_FLOOR, RANDOM_FLOOR, color=C["green"],
283                alpha=0.3, label="random floor")
284 ax.set_xlabel("unmodelled layer thickness (nm)")
285 ax.set_ylabel("thickness bias (nm)")
286 ax.set_title("(a) unmodelled layers")
287 ax.legend()
288
289 ax = axes[1]
290 ax.plot(dns, s3_bias, "o-", color=C["purple"], label="fitted bias")
291 ax.plot(dns, [x / nf_mid * D_TRUE for x in dns], "--", color=C["grey"],
292        label="$\\Delta n \\backslash d / n$")
293 ax.axhline(0, color=C["black"], lw=0.7)
294 ax.set_xlabel("film index offset $\\Delta n$")
295 ax.set_ylabel("thickness bias (nm)")
296 ax.set_title("(b) optical-thickness degeneracy")
297 ax.legend()
298
299 ax = axes[2]
300 labels = [b[0].split(" ", 1)[0] for b in budget]
301 vals = [abs(b[1]) for b in budget]
302 ax.bar(labels, vals, color=C["red"], alpha=0.8)
303 ax.axhline(RANDOM_FLOOR, color=C["green"], lw=1.4, label="random floor")
304 ax.set_yscale("log")
305 ax.set_ylabel("|bias| (nm)")
306 ax.set_title("(c) error budget")
307 ax.legend()
308 ax.grid(axis="x", visible=False)
309
310 save(fig, "exp04_systematics.png")
311
312 with open(f"{RESULTS}/exp04_results.json", "w") as f:
313     json.dump(res, f, indent=2)
314 print(f"\n    data -> results/exp04_results.json")

```

14.11 scripts/_style.py — 그림 스타일 (59줄)

```

1 """Shared plotting setup and path helpers for the experiment scripts."""
2
3 import os
4 import sys
5
6 import matplotlib
7 matplotlib.use("Agg")
8 import matplotlib.pyplot as plt          # noqa: E402
9
10 ROOT = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..")
11 RESULTS = os.path.join(ROOT, "results")
12 sys.path.insert(0, ROOT)
13 os.makedirs(RESULTS, exist_ok=True)
14
15 plt.rcParams.update({
16     "figure.dpi": 130,
17     "savefig.dpi": 130,
18     "savefig.bbox": "tight",
19     "font.size": 9,
20     "axes.grid": True,
21     "grid.alpha": 0.25,
22     "grid.linewidth": 0.6,
23     "axes.axisbelow": True,
24     "axes.titlesize": 10,
25     "axes.titleweight": "bold",
26     "legend.frameon": False,
27     "legend.fontsize": 8,
28     "lines.linewidth": 1.4,
29     "axes.spines.top": False,
30     "axes.spines.right": False,
31     "mathtext.default": "regular",
32 })
33
34 C = {

```

```

34 "blue": "#1f5fa8",
35 "red": "#c1382b",
36 "green": "#2e7d4f",
37 "orange": "#d97a1a",
38 "purple": "#6a4c93",
39 "grey": "#6b7280",
40 "black": "#1a1a1a",
41 }
42
43
44
45 def save(fig, name):
46     fig.tight_layout()
47     path = os.path.join(RESULTS, name)
48     fig.savefig(path)
49     plt.close(fig)
50     print(f"    figure -> results/{name}")
51     return path
52
53
54
55 def banner(title):
56     print()
57     print("=" * 78)
58     print(title)
59     print("=" * 78)

```

14.12 run_all.py — 전체 실행기 (52줄)

```

1  #!/usr/bin/env python3
2  """Run the full pipeline: validation suite, then all four experiments.
3
4  python run_all.py
5  """
6
7  import os
8  import subprocess
9  import sys
10 import time
11
12 ROOT = os.path.dirname(os.path.abspath(__file__))
13
14 STEPS = [
15     ("validation suite (34 checks)", ["tests/test_core.py"], ROOT),
16     ("exp01 forward model", ["exp01_forward_model.py"], os.path.join(ROOT, "scripts")),
17     ("exp02 inverse problem / ambiguity", ["exp02_inverse_ambiguity.py"], os.path.join(ROOT, "scripts")),
18     ("exp03 precision: CRLB vs Monte Carlo", ["exp03_precision_crlb.py"], os.path.join(ROOT, "scripts")),
19     ("exp04 systematic error budget", ["exp04_systematics.py"], os.path.join(ROOT, "scripts")),
20 ]
21
22
23
24 def main():
25     t_start = time.time()
26     failures = []
27     for label, args, cwd in STEPS:
28         print()
29         print("#" * 78)
30         print(f"# {label}")
31         print("#" * 78)
32         t0 = time.time()
33         r = subprocess.run([sys.executable] + args, cwd=cwd)
34         dt = time.time() - t0
35         status = "OK" if r.returncode == 0 else f"FAILED (exit {r.returncode})"
36         print(f"\n# {label}: {status} [{dt:.1f} s]")
37         if r.returncode != 0:
38             failures.append(label)
39
40     print()
41     print("=" * 78)
42     print(f"    total wall time: {time.time() - t_start:.1f} s")
43     if failures:
44         print(f"    FAILURES: {failures}")
45     else:
46         print("    all steps completed; see results/ for figures and JSON output")
47     print("=" * 78)
48     return 1 if failures else 0
49
50
51 if __name__ == "__main__":
52     sys.exit(main())

```

15 맺음말 — 이 문서에서 반복되는 하나의 주제

물리·코드·검증·실험을 관통하는 주제는 하나입니다.

무엇을 실제로 측정했는가, 그리고 그것이 무엇을 결정하는가.

- 프린지가 재는 것은 두께가 아니라 **광학 두께** $n \cdot d$ 입니다 $\rightarrow n$ 을 모르면 d 를 모릅니다.
- 단일 파장 반사율이 재는 것은 $\cos 2\delta$ **하나**입니다 $\rightarrow$ 차수와 측면이 축퇴합니다.
- CRLB가 제한하는 것은 **분산**입니다 $\rightarrow$ 편향은 따로 재야 합니다.
- 거칠은 막에서 “두께”는 **규약**입니다 $\rightarrow$ 완벽한 두 장비가 불일치합니다.

이 프로젝트가 방어할 수 있는 진술은 다음 하나입니다.

$\sigma_R = 2 \times 10^{-4}$ 의 분광 반사계로 217 nm SiO_2/Si 를 측정할 때, 적합기는 통계적으로 효율적이며(몬테카를로 산포가 세 자릿수 잡음 범위에서 Cramér-Rao 하한의 2% 이내) 랜덤 불확도는 4 pm입니다. 현실적 정확도는 약 1.3 nm이며, 모델 형태 — 모델링되지 않은 계면층, 거칠기 규약, 막 굴절률 가정 — 가 지배합니다. **결과를 개선하려면 더 나은 옵티마이저가 아니라 더 나은 시료 지식이 필요합니다.**

그리고 가장 큰 한계를 다시 명시합니다. **이 문서의 모든 숫자는 합성 데이터입니다.** 순방향 모델은 독립 구현체(Byrnes tmm)와 10^{-14} 수준으로 대조했지만, 역문제 파이프라인은 실제 시료를 만난 적이 없습니다. 이를 검증하려면 기준 교정된 산화막 웨이퍼가 필요합니다.